Expert Systems With Applications 332 (2027) 133614

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

SCALA-NIDS: Safety-constrained LLM-Advised closed-loop adaptation for online open-world network intrusion detection

Xiaojie Qin [a,b], Qingjun Yuan [a,b,c,*], Haopeng Fan [a,b], Jing Tao [c], Pinghui Wang [c], Jihong Teng [a], Siqi Lu [a,b], Yongjuan Wang [a,b]

[a] Information Engineering University, No. 62, Kexue Avenue, Zhengzhou, Henan, 450001, China
[b] Henan Key Laboratory of Network Cryptography Technology, Zhengzhou, Henan, 450001, China
[c] MoE Key Lab for Intelligent Networks and Network Security, Xi'an Jiaotong University, Xi'an, Shaanxi, 710049, China

ARTICLE INFO

Keywords:
Network intrusion detection
Online adaptation
Large language model
Open-world learning
Concept drift
Safety-constrained AI.

ABSTRACT

Network Intrusion Detection Systems (NIDS) in real-world environments face continuously evolving traffic streams caused by network reconfiguration, service migration, emerging attacks, and mixed distribution drift. Conventional closed-set and i.i.d.-based detectors often degrade under such online open-world conditions, while existing online adaptation methods usually lack coordinated drift perception, strategy decision-making, model updating, and safety validation. This paper proposes **Safety-Constrained LLM-Advised Adaptation for Network Intrusion Detection Systems (SCALA-NIDS)**, a closed-loop framework for online open-world NIDS. SCALA-NIDS follows a "Detection–Response–Validation" cycle by integrating runtime drift monitoring, constrained LLM-guided strategy reasoning, deterministic adaptation execution, and safety-gated validation. Given a structured Stats Snapshot, the LLM advisor selects high-level adaptation intents, which are mapped to predefined update configurations to ensure controllability and safety. SCALA-NIDS further introduces a strategy-driven multi-level teacher–student adaptation controller that coordinates sample selection, objective weighting, and model updating with source-data retention, target-sample selection, domain alignment, and consistency regularization. A Safety & Fallback Gate rejects or revokes risky updates. Experiments on UNSW-NB15 and NSL-KDD under temporally ordered online evaluation show that SCALA-NIDS achieves the highest accuracy (95.93%) and F1-score (94.42%) on UNSW-NB15, the highest F1-score (91.93%) on NSL-KDD, and the best average accuracy (93.18%) and F1-score (93.18%) across both datasets. Ablation and LLM-backend studies further confirm the effectiveness and practicality of the proposed framework.

1. Introduction

The deployment of Network Intrusion Detection Systems (NIDS) in real-world networks is an online operational process rather than a one-shot offline classification task. Conventional detectors are trained under closed-set and independent and identically distributed (i.i.d.) assumptions, yet practical environments routinely violate both. Network infrastructure evolves through reconfiguration and service migration, adversarial tactics shift over time, and novel attack types emerge beyond the training label space (European Union Agency for Cybersecurity, 2024). As illustrated in Fig. 1, an online open-world NIDS may encounter covariate shifts, unknown-class emergence, or their combination. A static detector under such conditions may misclassify unknown attacks as known categories or falsely flag benign distributional changes, eroding detection reliability (Zhang et al., 2025, 2024). Deep learning has also been increasingly applied to diverse traffic analysis tasks beyond conventional intrusion detection, including flow correlation analysis on anonymity networks (Huang et al., 2026), DNS tunnel detection (Gao et al., 2024), and adversarial website fingerprinting (Shen et al., 2024, 2025a).

Online adaptation has therefore become essential for maintaining NIDS performance. Existing approaches explore rule-triggered updating, test-time adaptation (TTA), drift detection, and continual learning (CL). TTA methods exploit unlabeled target data to reduce source–target distribution gaps (Niu et al., 2022, 2023; Wang et al., 2021, 2022b), while drift-aware and continual learning methods aim to detect distributional changes and preserve previously learned knowledge (De Lange et al., 2022; Zhang et al., 2025). Despite their progress, these methods address only individual aspects of online deployment: fixed thresholds do not generalize well across different drift intensities, online updates based

* Corresponding author.
E-mail addresses: xjqin24@163.com (X. Qin), gcxyuan@xjtu.edu.cn (Q. Yuan), haopengfan@outlook.com (H. Fan), jtao@mail.xjtu.edu.cn (J. Tao), phwang@xjtu.edu.cn (P. Wang), 1650063481@qq.com (J. Teng), 080lusiqi@sina.com (S. Lu), pinkywyj@163.com (Y. Wang).

https://doi.org/10.1016/j.eswa.2026.133614
Received 26 May 2026; Received in revised form 26 June 2026; Accepted 9 July 2026
Available online 10 July 2026
0957-4174/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

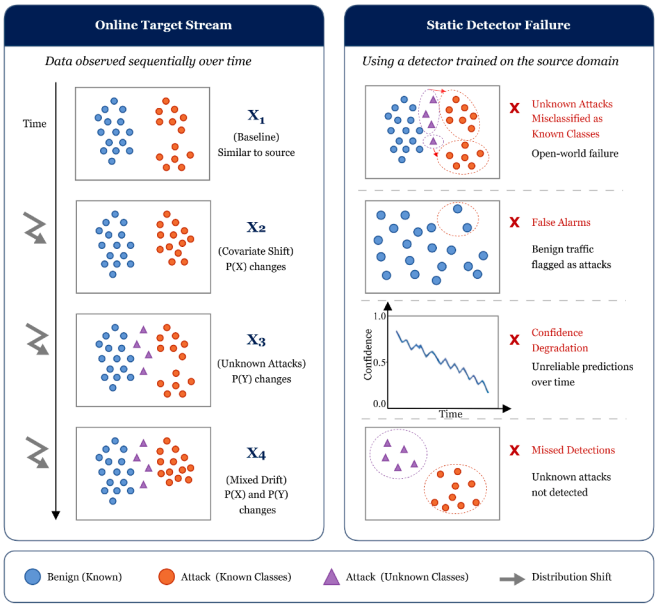


Fig. 1. Open-world NIDS under evolving target streams. The target stream may evolve from an initially similar distribution (X_1) to covariate shift (X_2), unknown-class emergence (X_3), or mixed drift (X_4), resulting in false alarms, unknown-attack misclassification, and confidence degradation.

on noisy unlabeled samples may propagate errors without explicit feedback, and most drift detectors are not designed to distinguish covariate shift from the emergence of previously unseen classes.

A reliable online open-world NIDS must therefore synthesize adaptation decisions from overlapping and often ambiguous runtime signals. The standard formulation treats adaptation as a local optimization step—selecting a loss term, tuning a learning rate, and applying an update—which is insufficient because it does not coordinate detection of distributional change, identification of its nature, selection of an appropriate response, and post-update verification of safety. Aggressive updates risk catastrophic forgetting (De Lange et al., 2022), while overly conservative ones allow detection performance to drift downward (Niu et al., 2023; Wang et al., 2022b). What is needed is a closed-loop design in which perception feeds into decision, decision into bounded execution, execution into validation, and validation feeds back into the next perception.

We address this gap with **Safety-Constrained LLM-Advised Adaptation for Network Intrusion Detection Systems (SCALA-NIDS)**. SCALA-NIDS treats online deployment as a "Detection–Response–Validation" cycle. It monitors runtime drift, invokes an LLM for strategy reasoning, executes the chosen strategy under deterministic constraints, and gates every candidate model update through a safety check before deployment. The Safety & Fallback Gate draws on trustworthy-AI principles around risk management, monitoring, and controllability (Standards, 2023).

The LLM operates at the strategy level rather than the optimization level. Instruction-tuned LLMs are effective at reasoning, planning, and tool use (Ouyang et al., 2022; Schick et al., 2023; Wei et al., 2022; Yao et al., 2023), which makes them well suited for cross-signal strategy selection. We provide the LLM with a compact structured summary, the Stats Snapshot, of the current runtime state, and require it to select a high-level adaptation strategy from a fixed predefined set. The LLM does not access raw traffic samples, does not generate hyperparameter values, and does not directly modify the model. Its output is processed by syntactic and semantic validation and then mapped, through a deterministic mapping module, to a predefined parameter template. This separation is intentional: the LLM contributes high-level reasoning, while the update pipeline remains bounded, reproducible, and auditable.

At execution time, SCALA-NIDS employs a teacher–student architecture to drive multi-level adaptation. The controller operates at three layers: sample selection, which determines the subset of target samples reliable enough to be used for adaptation; loss weighting, which balances source retention, domain alignment, pseudo-label learning, and consistency regularization; and model updating, which controls the magnitude of each parameter update. Teacher–student consistency and regularization have been shown effective for stabilizing continual and test-time adaptation under realistic distribution shifts (Niu et al., 2022, 2023; Wang et al., 2022b). By anchoring the student to a frozen teacher while allowing it to absorb new target patterns, SCALA-NIDS captures emerging distributional changes while preserving previously learned source-domain knowledge.

The main contributions of this paper are summarized as follows:

- **A safety-constrained closed-loop framework for online open-world NIDS.** SCALA-NIDS follows a "Detection–Response–Validation" cycle that ties together traffic detection, model adaptation, and safety validation. A Safety & Fallback Gate rejects or revokes risky updates based on performance feedback, improving robustness in dynamic environments.
- **A constrained LLM advisor with deterministic strategy-to-parameter mapping.** The LLM operates on structured Stats Snapshots, producing high-level adaptation recommendations. A deterministic mapping module turns these into predefined update configurations, preserving LLM reasoning while ruling out uncontrolled low-level optimization and privacy risks.
- **A strategy-driven multi-level teacher–student adaptation controller.** The controller adjusts sample selection and optimization objectives at the data, loss, and model levels, capturing target-domain distributional changes while mitigating source-domain knowledge forgetting.

2. Related work

2.1. Open-world network intrusion detection

Traditional network intrusion detection systems (NIDS) are commonly developed under closed-set and offline assumptions, where all attack categories are known during training and the deployment distribution is assumed to remain close to the training distribution. In real networks, however, traffic patterns evolve continuously due to changes in user behavior, service deployment, network topology, and adversarial activity, while new attack variants may emerge beyond the source-domain label space (European Union Agency for Cybersecurity, 2024). These properties make open-world traffic monitoring substantially more challenging than conventional static intrusion detection.

Open-world detection builds on open-set recognition, out-of-distribution detection, and novelty detection. Early studies introduced the notion of open space risk for rejecting samples outside known classes (Scheirer et al., 2013). OpenMax further modeled deep activation patterns with extreme value theory to improve open-set recognition (Bendale & Boult, 2016). In out-of-distribution detection, maximum softmax probability is a widely used baseline (Hendrycks & Gimpel, 2017), followed by methods based on temperature scaling and input perturbation (Liang et al., 2018), Mahalanobis distance (Lee et al., 2018), and energy-based scoring (Liu et al., 2020). More general formulations of open-set and generalized open-set recognition have also been extensively studied (Geng et al., 2021; Yang et al., 2024). On the NIDS architecture side, transformer-based frameworks such as Flow-Transformer (Manocchio et al., 2024) have demonstrated the effectiveness of capturing long-term traffic patterns through flexible component substitution across diverse network benchmarks.

For NIDS, however, identifying unknown or shifted traffic is more difficult because network data are heterogeneous, highly imbalanced, and temporally evolving. Unknown attacks may resemble known attack

families, while deep detectors may still produce overconfident predictions on unfamiliar traffic. Recent work addresses these challenges from complementary perspectives: variational autoencoders with Gaussian prototypes and noisy-label cleaning (Meng et al., 2025, 2026) detect unknown attacks in encrypted traffic, contrastive learning improves robustness against malicious traffic (Shen et al., 2025b), and cross-modal generative fusion exploits multimodal data (Wang et al., 2025). Pre-training-based approaches such as PCCA-Net (Chen et al., 2026) further enhance cross-scenario generalization by learning transferable representations from unlabeled traffic.

Recent evolving and online IDS methods partly address these issues through representation learning, uncertainty estimation, and continual adaptation. AOC-IDS (Zhang et al., 2024) combines contrastive representation learning, autoencoder-based modeling, and pseudo-label-based online updating. SSF (Zhang et al., 2025) addresses concept drift through strategic sample selection and forgetting. These methods provide strong baselines, but their adaptation is still driven mainly by predefined rules, sample-level filtering, or memory management, leaving the coordination of drift inspection, strategy selection, bounded adaptation, and post-update safety verification insufficiently explored.

2.2. Domain adaptation and test-time adaptation

Domain adaptation aims to transfer knowledge from a labeled source domain to a related target domain, often through feature alignment or distribution matching (Ganin et al., 2016; Long et al., 2015, 2018; Sun & Saenko, 2016). Karimian and Beigy (Karimian & Beigy, 2023) further formalize concept drift handling from a domain adaptation perspective by introducing multi-source feature alignment with theoretical generalization bounds for streaming data. More recently, test-time adaptation (TTA) has extended this idea by updating models during inference using unlabeled target data. Representative methods include TENT (Wang et al., 2021), which minimizes prediction entropy at test time; SHOT (Liang et al., 2020), which performs source-free adaptation; CoTTA (Wang et al., 2022b), which uses teacher–student consistency with stochastic restoration; EATA (Niu et al., 2022), which selects informative samples and regularizes updates; and SAR (Niu et al., 2023), which introduces sharpness-aware reliable entropy minimization (Liang et al., 2025).

Although these methods are effective in general distribution-shift settings, directly applying them to online NIDS is non-trivial. First, many methods assume a closed label space, whereas deployed traffic streams may contain unknown attacks or previously unseen behavior patterns. Second, many rely on a fixed adaptation policy or static hyperparameter settings, which may not be suitable for different drift conditions. Third, NIDS is safety-critical: an unsafe online update can persistently increase false alarms or missed detections. Therefore, online traffic anomaly detection requires not only adaptation itself, but also explicit strategy-level decision-making and safety-aware update control.

Compared with standard TTA pipelines, SCALA-NIDS treats online deployment as a closed-loop decision process. Rather than applying a single fixed adaptation policy, it selects a high-level adaptation strategy from a predefined strategy space according to runtime observations, maps that strategy to a bounded parameter template, and subjects the resulting candidate model to post-update safety validation. This design makes adaptation more controllable under streaming and open-world conditions.

2.3. LLMs for decision support and adaptive control

Large language models (LLMs) have demonstrated strong capabilities in reasoning, planning, instruction following, and tool use. GPT-3 (Brown et al., 2020) highlighted the effectiveness of in-context learning, while instruction tuning and reinforcement learning from human feedback improved controllability and alignment (Ouyang et al., 2022). Chain-of-thought prompting further showed that LLMs can support multi-step reasoning (Wei et al., 2022), and ReAct combined reasoning with external action execution (Yao et al., 2023). Related efforts such as Toolformer and autonomous-agent frameworks suggest that LLMs can serve as high-level planners or controllers in complex decision pipelines (Schick et al., 2023; Wang et al., 2024; Wu et al., 2023).

These properties have motivated increasing interest in using LLMs for decision support in cybersecurity (Xu et al., 2025; Yuan et al., 2025). Shafee et al. (2025) systematically evaluated LLM-based chatbots for OSINT-based cyber threat awareness, demonstrating both the potential and the current limitations of LLMs in security-critical applications. In an online adaptive NIDS setting, the system must reason over heterogeneous runtime evidence, including drift indicators, prediction confidence, prediction behavior, and recent safety feedback. Such cross-signal reasoning is difficult to encode with a single fixed heuristic, but is well aligned with the role of an LLM as a high-level strategy advisor.

At the same time, directly allowing an LLM to control low-level training or deployment is risky, especially in security-sensitive systems. LLM outputs may be non-deterministic, invalid, prompt-sensitive, or inconsistent with operational constraints. Prior studies and trustworthy AI guidelines have highlighted risks such as privacy leakage, memorization, hallucination, and unsafe tool use (Carlini et al., 2021; OWASP Foundation, 2025; Standards, 2023). For this reason, LLM-based adaptive systems should separate high-level reasoning from deterministic low-level execution.

SCALA-NIDS follows this principle. The LLM receives a structured runtime snapshot and outputs a high-level adaptation recommendation. This recommendation is then validated and mapped by deterministic control logic to a predefined parameter template, while the final candidate model is further checked by a safety gate before deployment. In this way, the framework uses the reasoning ability of LLMs without giving them unrestricted control over online model updates.

2.4. Summary and motivation

Existing studies have advanced open-world and adaptive NIDS through open-set detection, out-of-distribution analysis, continual learning, domain adaptation, and test-time adaptation. Methods such as AOC-IDS (Zhang et al., 2024), which combines contrastive representation learning, autoencoder-based modeling, and pseudo-label-based online updating, and SSF (Zhang et al., 2025), which addresses concept drift through strategic sample selection and forgetting, show that online updating, uncertainty control, and memory management can improve robustness under evolving traffic. These methods provide strong baselines, but their adaptation is still driven mainly by predefined rules, sample-level filtering, or memory management, leaving the coordination of drift inspection, strategy selection, bounded adaptation, and post-update safety verification insufficiently explored.

In existing work, adaptation is typically realized as a stand-alone update module rather than as part of a closed operational cycle, and drift detection, strategy selection, model updating, and safety verification are treated as separate components. As a result, a unified mechanism that jointly determines whether to adapt, how to adapt, and whether the resulting update is safe to deploy is largely missing. This structural gap motivates a closed-loop design in which perception, decision, execution, and validation are explicitly connected, with an LLM providing constrained strategy advice while deterministic control keeps the updates bounded and safe.

3. The proposed framework

We first define the problem setting, then describe the overall framework architecture, and finally present the details of each component.

3.1. Problem definition

We consider an open-world online NIDS problem in which the detector encounters evolving target-domain traffic after offline training. Let the source-domain supervised training set be

$$\mathcal{D}_s = \left\{\left(\mathbf{x}_i^s, y_i^s\right)\right\}_{i=1}^{N_s}, \qquad y_i^s \in \mathcal{Y}_s. \tag{1}$$

Before deployment, a detector

$$f_\theta : \mathcal{X} \to [0,1]^{|\mathcal{Y}_s|} \tag{2}$$

is trained on $\mathcal{D}_s$, producing a base model M_0:

$$\theta_0 = \arg\min_\theta \mathcal{L}_s\left(f_\theta; \mathcal{D}_s\right). \tag{3}$$

After deployment, unlabeled target traffic arrives continuously in chunks:

$$\mathcal{D}_t^{(k)} = \left\{\mathbf{x}_j^{t,k}\right\}_{j=1}^{N_t^{(k)}}, \qquad k = 1, 2, \ldots, K. \tag{4}$$

The underlying target distribution may change over time and may contain previously unseen traffic patterns that fall outside the source-domain label space. In this work, "open-world" characterizes such an evolving deployment environment in which both distribution shifts and unknown-class emergence may occur simultaneously. Rather than requiring explicit open-set classification at test time, SCALA-NIDS addresses this challenge holistically: unknown or shifted patterns are detected through anomaly scoring and multi-signal drift monitoring, and the model is adapted in a bounded manner when distributional changes are evidenced—without requiring that every novel traffic sample be individually labeled as "known" or "unknown."

For each incoming target chunk, the framework may generate a candidate model update:

$$\widetilde{\theta}_k = \mathcal{A}\left(\theta_{k-1}, \mathcal{D}_t^{(k)}\right), \tag{5}$$

where $\mathcal{A}(\cdot)$ denotes the online adaptation procedure. Since target labels are unavailable, the candidate update cannot be verified by direct target supervision. Therefore, it is deployed only if it satisfies a safety validation function:

$$\theta_k = \begin{cases} \widetilde{\theta}_k, & \text{if Safe}\left(\widetilde{\theta}_k; \theta_{k-1}, \mathcal{D}_t^{(k)}\right) = 1, \\ \theta_{k-1}, & \text{otherwise.} \end{cases} \tag{6}$$

The objective is to maintain reliable detection on evolving target traffic while preventing unsafe updates that may cause performance degradation, catastrophic forgetting, or unstable online behavior.

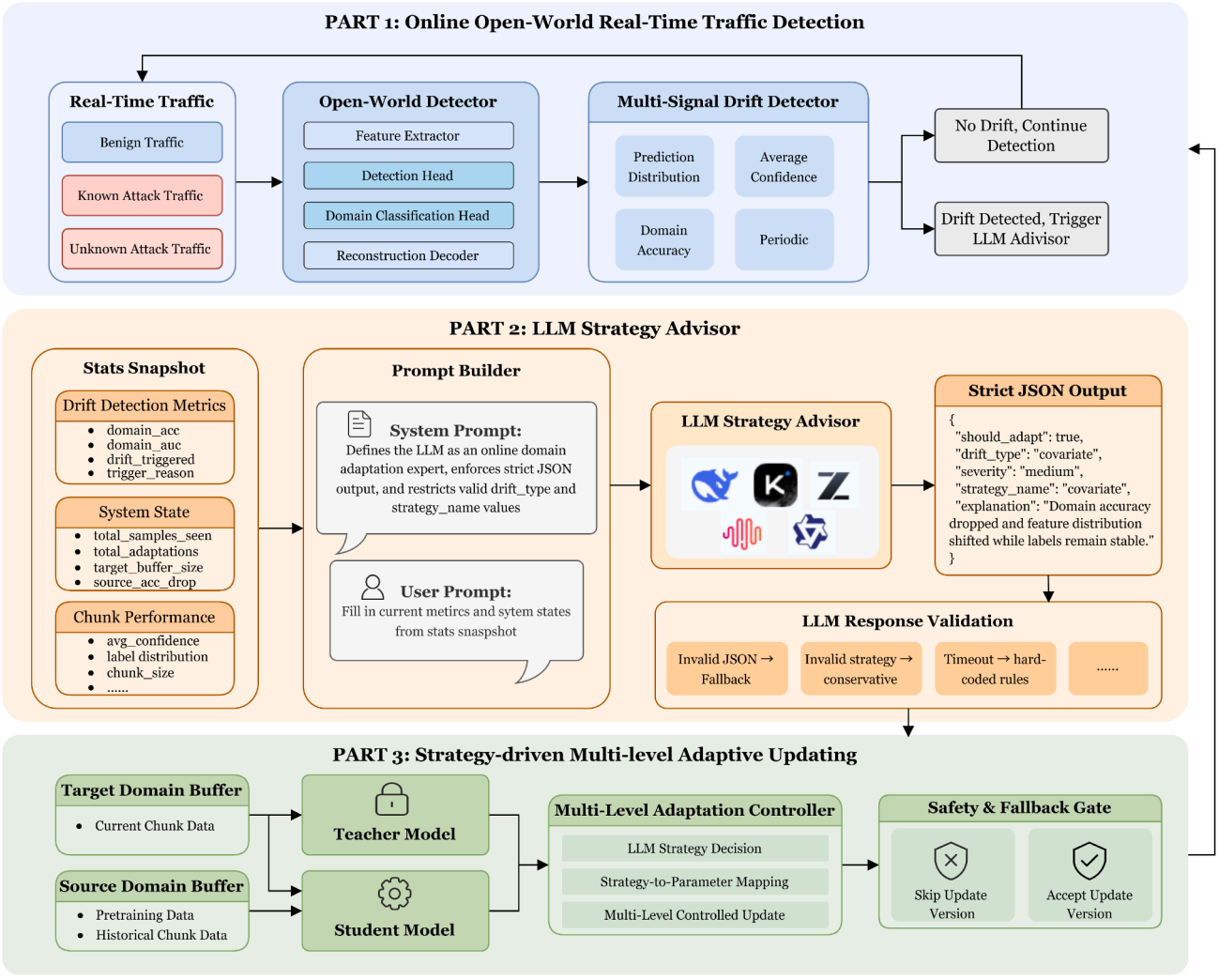


Fig. 2. Overview of the proposed SCALA-NIDS. The framework forms a closed-loop process that combines online detection, drift-aware inspection, strategy-level adaptation guidance, bounded model updating, and safety-controlled deployment.

Table 1
Key notations.

Notation	Meaning
$\mathcal{D}_s$, $\mathcal{D}_t^{(k)}$	Source data and the k-th target chunk.
f_{θ_k}, M_t	Detector and deployed model.
s_{anom}, s_{drift}	Anomaly score and drift-evidence flag.
$\mathbf{s}_t$, $\mathbf{p}_t$	Runtime statistics and LLM prompt.
π_t, π_t^{fb}	Final strategy decision and fallback decision.
$\mathcal{S}_{\text{str}}$, $\mathcal{G}$	Strategy set and drift-type set.
λ_t	Retrieved adaptation parameter template.
B_s, B_t^{safe}	Source buffer and safe target buffer.
M^T, M^S, $\widetilde{M}_{t+1}$	Teacher, student, and candidate model.
$\mathcal{L}_{\text{total}}$, Safe$(\cdot)$	Adaptation objective and safety gate.

3.2. Framework overview

Fig. 2 summarizes SCALA-NIDS as a safety-constrained "Detection–Response–Validation" loop for online open-world NIDS. For each incoming traffic chunk, the deployed detector produces anomaly predictions and runtime statistics. These statistics are monitored for distributional shift and summarized as a Stats Snapshot when adaptation may be needed.

When drift evidence is observed, the constrained LLM advisor selects a high-level adaptation strategy from a fixed strategy vocabulary. The LLM does not access raw traffic, generate hyperparameters, or update the model directly. Its output is first checked by syntactic and semantic validation; invalid, inconsistent, or unavailable outputs are replaced by a deterministic fallback decision.

The validated strategy is then mapped to a predefined parameter template that specifies sample selection, objective weights, and bounded optimization settings. The multi-level teacher–student adaptation module uses this template to produce a candidate model, which is deployed only if it passes the Safety & Fallback Gate. Otherwise, the previous verified model remains active. In this process, safety constraints are embedded directly into the loop: decisions are restricted to a finite strategy set, target samples are filtered before entering the adaptation buffer, source samples are retained to reduce forgetting, and candidate updates are bounded by fixed templates and conservative optimization rules. Thus, SCALA-NIDS keeps online adaptation controllable and reproducible without turning the LLM into an unconstrained optimizer. The key notations used throughout this framework are listed in Table 1.

3.3. Online open-world real-time traffic detection

The first stage of the framework handles real-time traffic detection alongside drift-aware monitoring. Its job is twofold: flag anomalous traffic in each incoming chunk, and decide whether the current traffic still lies within the distributional range that the deployed model can handle reliably. The distinction matters: a single anomalous prediction could be malicious, whereas a sustained distributional shift means the model itself is looking at unfamiliar patterns. We pair an open-world detector with a multi-signal drift detector to handle both cases.

3.3.1. Open-world detector

The open-world detector is pre-trained on the labeled source-domain training set before deployment. To avoid data leakage, only training samples are used during this stage. The pre-training objective is designed to provide three complementary capabilities. First, reconstruction learning on normal traffic allows the model to capture regular behavioral patterns. Second, supervised detection learning with normal and attack samples gives the model basic discriminative ability. Third, domain-aware training encourages the latent representation to be sensitive to potential source–target discrepancies. These capabilities are necessary because online target traffic may contain both known attacks and previously unseen patterns.

The detector contains a feature extractor, a detection head, a domain head, and a reconstruction decoder. Given an input traffic feature vector $\mathbf{x}$, the feature extractor produces a latent representation

$$\mathbf{z} = f_\theta(\mathbf{x}). \tag{7}$$

The detection head predicts the class posterior probability

$$\mathbf{q} = \text{softmax}(h_\phi(\mathbf{z})), \tag{8}$$

where q_1 denotes the anomaly probability in the binary detection setting. Meanwhile, the decoder reconstructs the input as $\hat{\mathbf{x}} = r_\psi(\mathbf{z})$, and the reconstruction error is computed by

$$e_{\text{rec}} = \frac{1}{d}\|\mathbf{x} - \hat{\mathbf{x}}\|_2^2. \tag{9}$$

We use both classification and reconstruction evidence to obtain the final anomaly score:

$$s_{\text{anom}} = \alpha \frac{e_{\text{rec}}}{\tau_{\text{rec}} + \epsilon} + (1-\alpha)q_1. \tag{10}$$

The motivation is that the two signals capture different types of abnormality. The classification score is effective for known attack patterns learned from labeled source data, while the reconstruction error is more sensitive to samples that deviate from normal traffic even if they do not match known attack classes. A sample is predicted as anomalous when

$$s_{\text{anom}} > \tau_{\text{anom}}. \tag{11}$$

In addition, the maximum posterior probability is used as a confidence indicator:

$$c = \max_{c'} q_{c'}. \tag{12}$$

Low confidence suggests that the sample lies near the decision boundary or outside familiar source-domain patterns. Therefore, confidence is not only useful for interpreting individual predictions, but also serves as an important signal for drift monitoring at the chunk level.

3.3.2. Multi-signal drift detector

Real-time anomaly detection alone is insufficient in online open-world environments, because the model may become unreliable when the target traffic distribution gradually changes. A single drift criterion is often unstable: feature-level tests may miss label-distribution changes, confidence degradation may be caused by temporary noisy samples, and prediction-distribution changes may not fully reflect latent feature shifts. For this reason, we use a multi-signal drift detector to monitor complementary forms of distributional deviation.

For the current traffic chunk $\mathcal{X}^{\text{cur}} = \{\mathbf{x}_i\}_{i=1}^N$, the model produces posterior predictions $\{\mathbf{q}_i\}_{i=1}^N$. From these predictions, we compute the average confidence $\bar{c}^{\text{cur}}$ and the empirical prediction distribution $\mathbf{p}^{\text{cur}}$. The former reflects whether the model is uncertain on the current chunk, while the latter captures whether the predicted class proportions differ from the reference distribution $\mathbf{p}^{\text{ref}}$ estimated from training data or a stable online period.

We also monitor feature-level domain separability using the domain head. If source and current target features can be easily distinguished in the latent space, the learned representation is no longer domain-invariant, which indicates a possible covariate shift. Let a_{dom} denote the domain discrimination accuracy. A larger a_{dom} implies stronger separability between source and target features.

Based on these observations, three evidence-driven drift signals are defined:

$$s_{\text{dom}} = \mathbb{I}\left(a_{\text{dom}} > \tau_{\text{dom}}\right), \tag{13}$$

$$s_{\text{conf}} = \mathbb{I}\left(\bar{c}^{\text{cur}} < \tau_{\text{conf}}\right), \tag{14}$$

$$s_{\text{dist}} = \mathbb{I}\left(\text{Test}\left(\mathbf{p}^{\text{cur}}, \mathbf{p}^{\text{ref}}\right) < \tau_{\text{dist}}\right). \tag{15}$$

Here, s_{dom} captures latent feature shift, s_{conf} captures prediction uncertainty, and s_{dist} captures prediction-distribution shift. The function

Test(·) denotes a statistical comparison between the current and reference prediction distributions, implemented with a chi-square test in our experiments.

The evidence-driven drift flag is then defined as

$$s_{\text{drift}} = \mathbb{I}\big(s_{\text{dom}} + s_{\text{conf}} + s_{\text{dist}} \geq 1\big). \tag{16}$$

This flag indicates whether the current chunk shows explicit evidence of distributional deviation.

In addition to evidence-driven drift detection, we introduce a periodic inspection signal

$$s_{\text{per}} = \mathbb{I}\big(t \bmod T_{\text{per}} = 0\big), \tag{17}$$

where t is the index of the current chunk. This signal is not treated as direct drift evidence. Instead, it serves as a conservative mechanism to inspect slowly accumulating changes that may not immediately violate any individual threshold.

Finally, the overall inspection flag is defined as

$$s_{\text{inspect}} = \mathbb{I}\big(s_{\text{drift}} + s_{\text{per}} \geq 1\big). \tag{18}$$

When $s_{\text{inspect}} = 1$, the current chunk is passed to the subsequent online handling module for further reasoning, buffering, or adaptation. Otherwise, the system continues normal online inference without triggering model update. This design separates anomaly prediction from adaptation triggering: anomalous samples are detected immediately, while model adaptation is activated only when there is sufficient drift evidence or scheduled inspection.

3.4. LLM strategy advisor

The LLM strategy advisor is introduced to separate high-level adaptation reasoning from low-level model optimization. In online open-world traffic detection, drift signals are often heterogeneous and ambiguous: a domain shift, a confidence drop, and a change in prediction distribution may imply different adaptation needs. Directly mapping these signals to model updates with fixed rules can be overly rigid, while allowing an LLM to generate training hyperparameters is unsafe and difficult to reproduce. Therefore, we use the LLM only as a strategy-level advisor. It receives compact runtime statistics, selects an adaptation strategy from a predefined strategy space, and its decision is further validated before being mapped to fixed parameter templates.

3.4.1. Runtime statistics and prompt construction

When the online detector triggers drift evidence or scheduled inspection, the framework summarizes the current system state into a runtime snapshot $\mathbf{s}_t$. This snapshot contains three types of information: drift indicators, model-state statistics, and chunk-level statistics. The drift indicators describe whether domain discrepancy, confidence degradation, or prediction-distribution shift has been observed. The model-state statistics record the current adaptation history and buffer status. The chunk-level statistics summarize the current traffic block, such as average confidence, predicted label distribution, and unknownness ratio.

Instead of sending raw traffic records to the LLM, we only provide aggregated statistics:

$$\mathbf{p}_t = \mathcal{P}(\mathbf{s}_t), \tag{19}$$

where $\mathcal{P}(\cdot)$ denotes the prompt-construction function. This design has two purposes. First, it reduces privacy exposure because the LLM does not access individual traffic samples. Second, it prevents the LLM from directly operating on detection instances, thereby restricting its role to strategy reasoning rather than sample-level prediction. A concrete instance of $\mathbf{p}_t$ is shown in Fig. 3, including the fixed system prompt (role, JSON schema, strategy semantics) and the dynamic user prompt populated with the runtime snapshot $\mathbf{s}_t$.

The prompt constrains the LLM to choose from a predefined strategy set

$$\mathcal{S}_{\text{str}} = \{\texttt{conservative}, \texttt{covariate}, \texttt{label}, \texttt{concept}, \texttt{mixed}\}, \tag{20}$$

```
=== SYSTEM PROMPT ===
You are an expert in network traffic drift analysis.
Your task is to recommend a model adaptation strategy
based on the runtime statistics provided.

You must respond with a strict JSON object containing
the following five fields:
- should_adapt:  boolean
- drift_type:    one of {none, covariate, label,
                         concept, mixed}
- severity:      one of {mild, moderate, severe}
- strategy_name: one of {conservative, covariate,
                         label, concept, mixed}
- explanation:   brief textual justification

Strategy semantics:
- conservative: only minor parameter updates;
  preferred when uncertain.
- covariate: feature-space adaptation for P(X) shift.
- label: pseudo-label refinement for P(Y) shift.
- concept: consistency regularization for P(Y|X) shift.
- mixed: combined adaptation for compound drift.

Rules:
- Prefer conservative when signals are weak/ambiguous.
- Choose mixed only when >=3 heterogeneous signals
  co-occur.
- Ground every decision in the provided drift metrics.

=== USER PROMPT ===
[Drift Metrics]
domain_accuracy:    0.72
avg_confidence:     0.81
distribution_pvalue:0.018
trigger_reasons:    ["domain_acc<0.75",
                     "pvalue<0.05"]

[System State]
source_accuracy:    0.94
buffer_target:      4500
buffer_source:      4500
strategy_history:   ["conservative","covariate"]

Please return your decision as a JSON object.
```

Fig. 3. A concrete example of the structured prompt sent to the LLM advisor. The system prompt is fixed across all calls; the user prompt is populated with the runtime snapshot $\mathbf{s}_t$.

and a drift-type set

$$\mathcal{C} = \{\texttt{none}, \texttt{covariate}, \texttt{label}, \texttt{concept}, \texttt{mixed}\}. \tag{21}$$

The strategy set specifies how the model should adapt, while the drift-type set describes why adaptation may be needed. This separation allows the LLM to express the cause of distributional change and the corresponding adaptation behavior in a structured manner.

3.4.2. LLM strategy decision and validation

Given the constructed prompt, the LLM produces a structured strategy decision:

$$\hat{\pi}_t^{\text{LLM}} = \mathcal{A}_{\text{LLM}}(\mathbf{p}_t), \tag{22}$$

where $\mathcal{A}_{\text{LLM}}(\cdot)$ denotes the LLM advisor. The output is required to follow a strict schema, including whether to adapt, the estimated drift type, the drift severity, the selected strategy, and the decision source. For compactness, the validated decision is denoted by π_t.

The key point is that the LLM decision is not directly trusted. Before it is used by the adaptation module, the response must pass both syntactic and semantic validation. Syntactic validation checks whether the response follows the required structured format. Semantic validation cross-references the LLM's claimed drift type with the observed drift signals: for instance, if only a `domain_accuracy` signal is triggered but the LLM claims `concept` drift, the decision is flagged as inconsistent and redirected to the fallback module. This cross-validation ensures that the LLM's reasoning remains grounded in the empirical evidence provided by the drift detector.

The final decision is defined as

$$\pi_t = \begin{cases} \hat{\pi}_t^{\text{LLM}}, & \text{if the LLM decision is valid,} \\ \pi_t^{\text{fb}}, & \text{otherwise,} \end{cases} \tag{23}$$

where π_t^{fb} is a rule-based fallback decision. This validation step forms a safety boundary between LLM reasoning and model adaptation. Even if the LLM response is unavailable, incorrectly formatted, or inconsistent

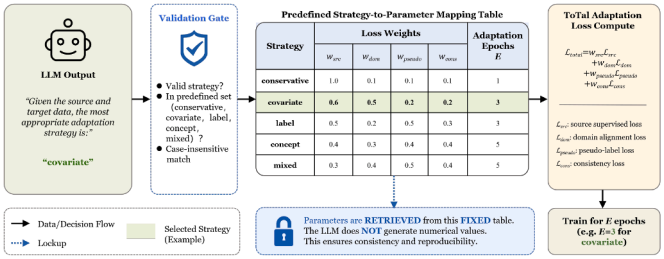


Fig. 4. LLM-guided strategy validation and parameter retrieval.

Table 2
Predefined strategy-to-parameter templates.

Strategy	w_{src}	w_{dom}	w_{pseudo}	w_{cons}	η	E	τ	γ
conservative	1.0	0.1	0.1	0.1	1×10^{-4}	1	0.99	3:1
covariate	0.6	0.5	0.2	0.2	5×10^{-4}	3	0.95	2:1
label	0.5	0.2	0.5	0.3	5×10^{-4}	3	0.85	1:1
concept	0.4	0.3	0.4	0.4	1×10^{-3}	5	0.90	1:2
mixed	0.3	0.4	0.5	0.4	1×10^{-3}	5	0.85	1:2

with the runtime evidence, the framework can still produce a deterministic and bounded decision. Owing to the strict JSON schema, the finite strategy and drift-type sets, and the semantic validator, the final decision π_t is determined primarily by the runtime statistics $\mathbf{s}_t$ rather than by surface-level prompt phrasing.

3.4.3. Fallback strategy

The fallback strategy is used when the LLM is unavailable or its output fails validation:

$$\pi_t^{\text{fb}} = \text{FallbackRules}(\mathbf{s}_t). \tag{24}$$

Fallback is triggered by API timeout, temporary unavailability, rate limitation, invalid output format, or context-length overflow. When any of these occurs, the system immediately switches to the rule-based module without retrying.

The fallback module follows a conservative principle. If no explicit drift evidence is observed, it avoids unnecessary adaptation or selects the `conservative` strategy. If the main evidence comes from feature-level domain discrepancy or prediction-distribution shift, the drift is treated as covariate-related. If the unknownness ratio increases, the system tends to select a label-space adaptation strategy. If confidence degradation dominates, the drift may indicate a change in the decision boundary and is therefore handled as potential concept drift. When multiple signals are activated simultaneously, the fallback module selects the `mixed` strategy.

The purpose of the fallback module is not to outperform the LLM, but to guarantee availability and safety. It ensures that the online system does not depend solely on the LLM and that every triggered inspection can be handled by a reproducible decision rule. Since the fallback decision shares the same representation as the LLM decision, the downstream adaptation module can process both types of decisions uniformly.

3.4.4. Strategy-to-parameter mapping

After validation or fallback, the selected high-level strategy is mapped to a predefined parameter template:

$$\lambda_t = \mathcal{M}(\pi_t), \tag{25}$$

where $\mathcal{M}(\cdot)$ is a deterministic mapping function. As shown in Fig. 4, the LLM only determines the adaptation strategy and never generates numerical hyperparameters. This design keeps online adaptation bounded, reproducible, and auditable.

The retrieved template specifies the loss weights, learning rate, number of adaptation epochs, pseudo-label threshold, and source/target sampling ratio. The predefined templates are shown in Table 2.

These templates encode different adaptation behaviors. The `conservative` strategy emphasizes source-knowledge retention and uses a high pseudo-label threshold with a small learning rate. The `covariate` strategy increases the weight of domain alignment to handle feature-distribution shift. The `label` strategy relies more on pseudo-label learning to accommodate emerging label-space patterns. The `concept` strategy strengthens consistency regularization and target-domain learning to adjust the decision boundary. The `mixed` strategy combines stronger target learning with moderate source replay for complex drift.

By separating strategy selection from parameter assignment, the framework benefits from the LLM's reasoning ability while avoiding unconstrained LLM control over optimization. Thus, the LLM can guide online adaptation, but the actual update remains governed by deterministic and preverified templates. The five strategies correspond one-to-one to the drift scenarios in Section 3.2, and the parameter values were obtained through a preliminary tuning study on a held-out validation split prior to the online evaluation.

3.5. Strategy-driven multi-level adaptive updating

After the strategy decision π_t is validated and mapped to the parameter template λ_t, the framework performs bounded online adaptation. This module does not decide *whether* or *how* to adapt at the strategy level; instead, it executes the selected adaptation behavior under explicit data, optimization, and model constraints. The goal is to improve target-domain fitness without allowing unreliable target samples or overly aggressive updates to corrupt the deployed detector. To this end, we combine buffer-based sample selection, teacher–student learning, and bounded optimization.

3.5.1. Source and target domain buffers

Online target adaptation faces a fundamental trade-off: the model must learn from new traffic patterns, but excessive reliance on recent target samples may quickly amplify pseudo-label noise and cause catastrophic forgetting. We therefore maintain both a source buffer and a target buffer.

The source buffer, denoted by B_s, stores source-domain samples used for pretraining and source-knowledge replay. Its role is to anchor the adaptation process to previously learned discriminative structure, so that online updates do not drift too far from the original detection capability.

The target buffer stores recent target-domain samples that are considered sufficiently reliable for adaptation. Rather than admitting all incoming target samples, we only keep samples with low unknownness and high prediction confidence:

$$B_t = \left\{ \mathbf{x}_i^t \mid u_i < \tau_b,\ c_i > \tau_c \right\}, \tag{26}$$

where u_i is the unknownness score and c_i is the prediction confidence. This filtering step prevents clearly uncertain or highly unfamiliar samples from directly influencing model updates.

To further suppress noisy pseudo-label learning, we apply an entropy-based reliability filter before adaptation. The resulting safe target buffer is

$$B_t^{\text{safe}} = \left\{ \mathbf{x}_i^t \in B_t \mid H(\mathbf{x}_i^t) < \tau_e \right\}, \tag{27}$$

where $H(\mathbf{x}_i^t)$ is the prediction entropy. Only samples in B_t^{safe} are used as target-domain adaptation candidates. At each update step, mini-batches are drawn from the source buffer and the safe target buffer:

$$\mathcal{B}_s \sim B_s, \qquad \mathcal{B}_t \sim B_t^{\text{safe}}. \tag{28}$$

This buffer design acts as the data-level safety mechanism of the framework. The source buffer preserves historical knowledge, while the safe target buffer ensures that adaptation is driven by recent yet relatively trustworthy target evidence rather than arbitrary online traffic.

3.5.2. Teacher–student adaptation objective

Directly fine-tuning the deployed model on target samples is risky in the online setting, because target supervision is unavailable and pseudo-label errors may accumulate rapidly. We therefore adopt a teacher–student adaptation scheme. When adaptation is triggered, the current deployed model M_t is copied into a frozen teacher M^T and a trainable student M^S:

$$M^T \leftarrow M_t, \qquad M^S \leftarrow M_t. \tag{29}$$

The teacher provides stable pseudo-supervision, while the student absorbs new target information under bounded updates. This separation reduces the risk of self-reinforcing drift that often occurs when a single model both generates and learns from its own noisy predictions.

For target samples in $\mathcal{B}_t$, the teacher generates pseudo-labels, and only sufficiently confident target predictions are used for pseudo-label learning:

$$\mathcal{B}_{\mathrm{pl}} = \left\{ \mathbf{x}_i^t \in \mathcal{B}_t \mid q_i^T > \tau_t \right\}, \tag{30}$$

where q_i^T is the teacher confidence and τ_t is determined by the selected strategy template. This confidence threshold provides a second safeguard against noisy target supervision.

The student is optimized with a multi-term objective:

$$\mathcal{L}_{\mathrm{total}} = w_{\mathrm{src}}\mathcal{L}_{\mathrm{src}} + w_{\mathrm{dom}}\mathcal{L}_{\mathrm{dom}} + w_{\mathrm{pseudo}}\mathcal{L}_{\mathrm{pseudo}} + w_{\mathrm{cons}}\mathcal{L}_{\mathrm{cons}}. \tag{31}$$

Each term addresses a different aspect of safe online adaptation.

The source retention loss $\mathcal{L}_{\mathrm{src}}$ keeps the student aligned with source-domain supervision, thereby mitigating catastrophic forgetting. The domain alignment loss $\mathcal{L}_{\mathrm{dom}}$ reduces feature-level discrepancy between source and target samples, which is especially important under covariate drift. The pseudo-label loss $\mathcal{L}_{\mathrm{pseudo}}$ allows the student to absorb target-specific information from reliable target predictions. The consistency loss $\mathcal{L}_{\mathrm{cons}}$ encourages prediction stability under perturbations, which improves robustness when target labels are unavailable.

For clarity, these terms are instantiated as

$$\mathcal{L}_{\mathrm{src}} = -\frac{1}{|\mathcal{B}_s|} \sum_{(\mathbf{x}_i^s, y_i^s)\in\mathcal{B}_s} \log p_{\theta_S}(y_i^s \mid \mathbf{x}_i^s), \tag{32}$$

$$\mathcal{L}_{\mathrm{pseudo}} = -\frac{1}{|\mathcal{B}_{\mathrm{pl}}| + \epsilon} \sum_{\mathbf{x}_i^t\in\mathcal{B}_{\mathrm{pl}}} \log p_{\theta_S}(\hat{y}_i^T \mid \mathbf{x}_i^t), \tag{33}$$

and

$$\mathcal{L}_{\mathrm{cons}} = \frac{1}{|\mathcal{B}_t| + \epsilon} \sum_{\mathbf{x}_i^t\in\mathcal{B}_t} \left\| p_{\theta_S}(\cdot \mid a_1(\mathbf{x}_i^t)) - p_{\theta_T}(\cdot \mid a_2(\mathbf{x}_i^t)) \right\|_2^2, \tag{34}$$

where $a_1(\cdot)$ and $a_2(\cdot)$ denote two perturbations of the same target sample. The domain alignment term $\mathcal{L}_{\mathrm{dom}}$ is implemented through standard adversarial domain adaptation with a gradient reversal layer.

The contribution of each loss term is controlled by the strategy-specific template λ_t. This is important because different drift types require different adaptation emphasis. For example, covariate drift benefits more from domain alignment, while label-space or concept-related drift requires stronger pseudo-label learning and consistency regularization.

To keep the online update bounded, the template also specifies the learning rate, the number of adaptation epochs, and the source–target sampling ratio. In addition, we apply conservative optimization constraints such as partial layer freezing and gradient clipping:

$$\nabla\theta_{\mathrm{shallow}} = 0, \qquad \left\| \nabla_{\theta_S}\mathcal{L}_{\mathrm{total}} \right\|_2 \le G_{\max}. \tag{35}$$

These constraints reduce the chance that transient noisy chunks cause abrupt representation drift. After optimization, the updated student becomes the candidate model:

$$\widetilde{M}_{t+1} \leftarrow M^S. \tag{36}$$

Overall, this module enforces bounded adaptation at three levels. At the data level, only filtered target samples are used. At the supervision level, pseudo-label learning is mediated by a frozen teacher. At the optimization level, the update strength is limited by predefined templates and conservative training constraints. This design allows the framework to adapt online while keeping the update process stable and controllable.

3.5.3. Safety & fallback gate

The adapted student model is not deployed immediately. Instead, the candidate model $\widetilde{M}_{t+1}$ must pass a Safety & Fallback Gate before replacing the current deployed model M_t. This module serves as the final safeguard of the framework: even if the drift signal is reasonable, the strategy decision is valid, and the bounded adaptation procedure has been executed as intended, the resulting update may still be harmful. Therefore, model replacement is determined by post-update safety verification rather than by adaptation completion alone.

The deployment rule is defined as

$$M_{t+1} = \begin{cases} \widetilde{M}_{t+1}, & \text{if } \mathrm{Safe}\left(\widetilde{M}_{t+1}; M_t, D_s^{\mathrm{val}}, B_t^{\mathrm{safe}}\right) = 1, \\ M_t, & \text{otherwise}, \end{cases} \tag{37}$$

where D_s^{val} is the source-domain validation set and B_t^{safe} is the current safe target buffer.

This gate is conceptually different from the validation step in Section 3.4. The earlier validation checks whether the *decision* is well-formed and admissible, whereas the Safety & Fallback Gate checks whether the *resulting model* is safe to deploy. In other words, the former constrains strategy selection, while the latter constrains model replacement.

To judge whether the candidate model is acceptable, we evaluate it from several complementary perspectives:

$$\mathrm{Safe}\left(\widetilde{M}_{t+1}\right) = \mathbb{I}\left(\bigwedge_{j=1}^{5} c_j\left(\widetilde{M}_{t+1}, M_t\right) \right), \tag{38}$$

where the criteria jointly assess optimization stability, source-knowledge preservation, target-buffer reliability, prediction-collapse avoidance, and unknownness stability.

First, the update should remain optimization-stable. If the post-update adaptation loss increases sharply relative to the pre-update model, the update is likely unstable rather than beneficial:

$$c_1: \quad \frac{\mathcal{L}_{\mathrm{total}}\left(\widetilde{M}_{t+1}; \mathcal{B}_s, B_t^{\mathrm{safe}}\right)}{\mathcal{L}_{\mathrm{total}}\left(M_t; \mathcal{B}_s, B_t^{\mathrm{safe}}\right) + \epsilon} \le \tau_{\mathrm{loss}}. \tag{39}$$

Second, adaptation should not excessively damage source-domain capability. Since the deployed detector must retain its previously learned discrimination ability, we require that source-domain validation performance does not drop beyond a tolerance:

$$c_2: \quad \mathrm{Acc}\left(\widetilde{M}_{t+1}; D_s^{\mathrm{val}}\right) \ge \mathrm{Acc}\left(M_t; D_s^{\mathrm{val}}\right) - \delta_{\mathrm{src}}. \tag{40}$$

Third, the candidate model should not become less reliable on the safe target buffer. We therefore monitor whether the average prediction confidence on trusted target samples degrades substantially:

$$c_3: \quad \bar{c}\left(\widetilde{M}_{t+1}; B_t^{\mathrm{safe}}\right) \ge \bar{c}\left(M_t; B_t^{\mathrm{safe}}\right) - \delta_{\mathrm{conf}}. \tag{41}$$

Fourth, we explicitly guard against prediction collapse. An update that makes the model output nearly degenerate predictions on the target buffer is unsafe even if its training loss decreases. We therefore require sufficient entropy in the predicted class distribution:

$$c_4: \quad H\left(\widetilde{M}_{t+1}; B_t^{\mathrm{safe}}\right) > H_{\min}. \tag{42}$$

Fifth, the update should not induce an abrupt change in the unknownness profile of the target buffer. A large shift in the proportion of high-unknownness samples may indicate that the adapted model has become unstable or overly biased:

$$c_5: \quad \left| \rho_{\mathrm{unk}}\left(\widetilde{M}_{t+1}; B_t^{\mathrm{safe}}\right) - \rho_{\mathrm{unk}}\left(M_t; B_t^{\mathrm{safe}}\right) \right| < \delta_{\mathrm{unk}}. \tag{43}$$

These checks are intentionally heterogeneous. No single metric is sufficient to guarantee safe deployment: loss reduction alone may hide collapse, source preservation alone may ignore target usefulness, and target confidence alone may be misleading under overconfident errors. By combining multiple criteria, the gate evaluates whether the candidate model is not only adaptive, but also stable and deployable.

If any criterion is violated, the update is rejected and the system keeps the previous model M_t. The gate also supports rollback: if a newly accepted model later shows sustained degradation over subsequent chunks, the system reverts to the last verified stable checkpoint. In addition, rejection or rollback information is fed back into the next runtime snapshot. This closes the adaptation loop, allowing the LLM advisor or the fallback rules to choose a more conservative strategy in later rounds when previous updates have proven unsafe.

4. Experiments

4.1. Experimental objectives

The experiments are designed to answer the following research questions:

- **RQ1 (Overall Performance)**: Does SCALA-NIDS achieve superior and more stable detection performance compared with representative online, continual, and traditional baseline methods in evolving open-world environments?
- **RQ2 (LLM Backend Analysis)**: How do different LLM backends vary in terms of adaptation effectiveness, inference latency, token cost, and call reliability?
- **RQ3 (Strategy and Explanation Quality)**: Can the LLM advisor generate reasonable strategy recommendations and provide high-quality, interpretable explanations grounded in the observed drift indicators?
- **RQ4 (Ablation Study)**: How do the individual components, including the LLM-guided strategy selection, the multi-level teacher–student controller, and the Safety & Fallback Gate, contribute to the overall framework?
- **RQ5 (Parameter Sensitivity)**: How do key hyperparameters, including chunk size, pseudo-label threshold, source buffer size, gradient clipping, and teacher EMA coefficient, influence the system's adaptation stability and detection capability?

4.2. Datasets

We evaluate the proposed framework on public network intrusion detection datasets, including NSL-KDD (Tavallaee et al., 2009) and UNSW-NB15 (Moustafa & Slay, 2015). Each dataset contains benign traffic and multiple attack categories. NSL-KDD is a refined version of the KDD'99 benchmark (UCI Machine Learning Repository, 1999) and has been widely used for evaluating intrusion detection models due to its reduced redundancy and balanced difficulty levels (Tavallaee et al., 2009). UNSW-NB15 was generated using the IXIA PerfectStorm tool to provide more modern normal and attack traffic patterns, including multiple contemporary attack families (Moustafa & Slay, 2015).

4.3. Baselines

To evaluate the effectiveness of our method in online intrusion detection and continual adaptation, we compare it with representative online/continual IDS methods, contrastive learning methods, and traditional machine learning baselines.

Specifically, AOC-IDS (Zhang et al., 2024) is an autonomous online IDS that combines an autoencoder with cluster-repelled contrastive loss and uses Gaussian fitting for adaptive decision making. SSF (Zhang et al., 2025) addresses concept drift through K-S test-based drift detection, sample selection, and forgetting.

Table 3
Comparison with baseline methods on the UNSW-NB15 and NSL-KDD datasets.

Method	UNSW-NB15				NSL-KDD			
	Acc	Pre	Recall	F1	Acc	Pre	Recall	F1
SCALA-NIDS	**95.93**	**93.25**	96.86	**94.42**	90.42	**89.79**	94.60	**91.93**
AOC-IDS	89.19	90.65	89.70	90.14	88.90	85.99	**96.21**	90.81
SSF	90.27	89.06	93.89	91.40	**90.50**	89.22	94.79	91.90
FeCo	72.50	91.18	55.41	68.93	81.10	90.72	74.44	81.72
CIDS	82.61	78.91	96.28	86.03	82.29	93.54	77.18	80.65
DTC	85.95	82.32	94.86	88.15	78.98	92.27	69.28	78.92
RF	85.93	80.25	**98.75**	88.55	76.74	96.71	61.23	74.99
XGBoost	86.97	82.85	98.29	89.26	77.95	96.46	63.60	76.62

We also include contrastive learning baselines, including FeCo (Wang et al., 2022a) and CIDS (Yue et al., 2022). FeCo learns discriminative representations using InfoNCE loss with a fixed detection threshold, while CIDS combines contrastive loss and cross-entropy loss for intrusion classification. In addition, traditional machine learning baselines, including decision tree classifier (DTC) (Quinlan, 1993), random forest (RF) (Breiman, 2001), and XGBoost (Chen & Guestrin, 2016), are used for comparison. These methods learn fixed decision boundaries from offline training data and serve as standard non-continual baselines. The evaluation protocol, including dataset splits and preprocessing, follows AOC-IDS (Zhang et al., 2024). Baseline results for AOC-IDS and the other methods are cited from (Zhang et al., 2024), while SSF results are from its original paper (Zhang et al., 2025).

4.4. Training and adaptation protocol

The detector is trained in three stages: (1) **Domain-head warm-up**: The feature extractor is frozen, and the domain classifier g_ω is trained (3 epochs) to distinguish source anchors from perturbed simulated target samples. (2) **Reconstruction-only**: The encoder–decoder is trained (5 epochs) on benign traffic using a feature-weighted MSE loss (weights: 3.0 for numerical, 0.3 for categorical features). (3) **Joint training**: The model is optimized using:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + 0.4\mathcal{L}_{\text{compact}} + 0.3\mathcal{L}_{\text{ce}} + 0.1\mathcal{L}_{\text{domain}}. \tag{44}$$

$\mathcal{L}_{\text{compact}}$ tracks the feature center via EMA (0.9 momentum). $\mathcal{L}_{\text{ce}}$ is computed on balanced samples with the encoder detached. $\mathcal{L}_{\text{domain}}$ employs a gradient reversal layer with $\lambda_{\text{grl}} = \min(1, t/10)$, where t is the epoch.

Implementation Details. We use dataset-specific MLP architectures: UNSW-NB15 ([512, 256] hidden layers) and NSL-KDD ([256, 256]). Online adaptation uses Adam/AdamW, a teacher model updated via EMA (0.99), and buffers of size 2000. Shallow encoder layers are frozen during adaptation to preserve source representations. The LLM advisor (temperature 0.3) is accessed via API with response caching and error-handling fallbacks.

Evaluation Paradigm. The test stream is divided into temporally ordered chunks (size 4000 for UNSW-NB15, 4500 for NSL-KDD). For each chunk, the model is first evaluated and then updated ("test-then-train"). Reported metrics are averaged over all chunks to reflect both steady-state performance and adaptation stability.

4.5. Comparison with baseline methods

Table 3 reports the online evaluation results. All metrics are averaged over temporal chunks, so they reflect both per-chunk detection quality and long-term adaptation stability.

On **UNSW-NB15**, the gap between SCALA-NIDS and the baselines is substantial. Our method records the highest Accuracy (95.93%), Precision (93.25%), and F1-score (94.42%). Relative to the strongest online baseline, AOC-IDS, this represents a 6.74% gain in Accuracy and a 4.28% gain in F1. Traditional supervised models (RF, XGBoost) push

Table 4
LLM calling cost and quality summary of different models (NSL-KDD dataset).

Model	Call	Total(s)	Mean(s)	Median(s)	Std	Min	Max	P_Tok_Sum	C_Tok_Sum	T_Tok_Sum	Att	Succ	Err	FB
DeepSeek-V4-Flash	6	57.9	9.65	11.23	5.96	1.79	15.74	3467	3110	6577	1.0	6	0	0
GLM-5.1	6	321.1	53.52	43.42	32.32	27.54	119.19	3339	7413	10,752	1.2	5	0	0
Kimi-K2.6	6	1670.7	278.45	278.47	0.14	278.20	278.62	0	0	0	3.0	0	6	6
MiniMax-M2.5	6	59.7	9.95	8.36	2.93	6.95	15.19	3424	4506	7930	1.0	6	0	0
Qwen3.5-397B	6	1658.5	276.41	278.25	4.07	267.32	278.34	591	571	1162	3.0	0	5	6

Recall slightly higher (~98%), but their Precision drops to 80%–82%, which in practice means an unmanageable false-alarm rate. SCALA-NIDS keeps Recall at 96.86% while holding Precision above 93%–a much more operational trade-off.

On **NSL-KDD**, SCALA-NIDS posts the highest F1-score (91.93%) and Precision (89.79%). SSF edges ahead by 0.08% in Accuracy on the back of aggressive Recall, but gives up Precision in return. SCALA-NIDS outperforms AOC-IDS on Accuracy, Precision, and F1-score, underscoring that the closed-loop adaptation remains effective even when the distribution shift is less dramatic but steady.

Taken together, the results point to two strengths:

1. **Robustness under distribution shift.** Conventional supervised models (RF, XGBoost) and static deep detectors (CIDS, FeCo) lose considerable ground on UNSW-NB15, where the shift is more severe. SCALA-NIDS stays near its peak through active, bounded adaptation.
2. **Precision-oriented adaptation.** Where baselines trade Precision for Recall–sometimes sharply–the LLM-guided strategy selection prevents the model from collapsing toward a biased representation. High Precision is preserved across diverse attack scenarios.

4.6. Comparison of different large language models

This experiment aims to evaluate the impact of different LLMs as strategy advisors on the framework performance. We compare multiple representative LLMs, including DeepSeek-V4-Flash, GLM-5.1, Kimi-K2.6, MiniMax-M2.5, and Qwen3.5-397B. To ensure a fair comparison among different LLM backends, this experiment is conducted only on the NSL-KDD dataset, and the chunk size is fixed to 4500, consistent with the main evaluation protocol. This setting aligns the experimental conditions and isolates the effect of the LLM advisor itself from other factors such as dataset variation and chunk-size configuration.

For each model, we record the calling latency, token consumption, retry behavior, error frequency, fallback activation, and the distribution of selected adaptation strategies. The goal is to analyze not only the final detection performance, but also the practical deployment cost, reliability, and explanation quality of different LLM backends under the same data stream setting.

4.6.1. Calling cost and reliability

Table 4 and Fig. 5 jointly characterize the calling cost and reliability of the five LLM backends on the NSL-KDD dataset.

DeepSeek-V4-Flash (DS-V4F). DS-V4F is fully reliable, with all six calls succeeding on the first attempt and zero fallback triggers. Its per-call latency is low but exhibits notable inter-chunk variability: the fastest call completes in 1.8 s (chunk #2), the slowest in 15.7 s (chunk #3), and most calls cluster between 2 s and 16 s, for an overall mean of 9.65 s. Prompt token consumption is highly consistent across chunks (520–560, a spread of only ~40), reflecting near-identical structured snapshot input length. Completion tokens, however, vary dramatically: from as few as 90–120 in chunks #1, #2, and #4, to 800–960 in chunks #0, #3, and #5. This bimodal output pattern indicates that DS-V4F occasionally produces extensive justifications when drift signals are stronger, but otherwise defaults to terse, decision-only outputs.

MiniMax-M2.5 (MM-M2.5). MM-M2.5 is also fully reliable, achieving 6/6 first-attempt successes with zero fallback. Its latency profile is the most stable among all models: the majority of calls fall within 7–13 s (mean 9.95 s), with only one outlier at 15.2 s (chunk #4). Its prompt token consumption mirrors that of DS-V4F (510–550 across chunks). Completion token usage occupies a consistent middle ground: between 550 and 890 tokens per call, with no extreme lows or highs. This balanced resource profile corroborates the moderate but steady strategy diversity observed in the next subsection.

GLM-5.1. GLM-5.1 exhibits partial reliability: only one of six calls requires a retry, and all six calls eventually succeed without triggering fallback. Its latency, however, is the highest among all models by a considerable margin. Slow calls reach 57.7–119.2 s (chunks #4, #5, #1), while even its fastest calls (chunks #0, #2, #3) require 27–30 s. The mean latency of 53.52 s is approximately 5× that of the two fully reliable models. Prompt tokens are not significantly elevated relative to the other backends (500–540 across chunks), ruling out longer input as the cause. The source of the latency is therefore on the output side: GLM-5.1 generates 1050–1200 completion tokens per call—nearly double the median output of MM-M2.5 and almost an order of magnitude more than the low-output chunks of DS-V4F. This consistently verbose generation pattern inflates both response time and total token consumption (10,752 total, the highest across all models).

Kimi-K2.6 and **Qwen3.5-397B** did not yield accepted LLM-driven decisions in this evaluation. Kimi-K2.6 times out on every call (~278 s per attempt, zero prompt and zero completion tokens), so no LLM response is obtained for any chunk. Qwen3.5-397B records one response attempt but returns errors in the remaining calls, and the recorded response is not accepted as an executable strategy decision after timeout/error handling and validation. In our implementation, the API timeout threshold is set to 120 s; Kimi-K2.6 consistently exceeds this limit, while Qwen3.5-397B is affected by timeout/error handling and unaccepted output. Consequently, all six chunks are handled by the fallback module for both backends, and these two models are therefore excluded from the per-call resource visualization in Fig. 5.

Summary of trade-offs. Across the three models with successful outputs, prompt token consumption is nearly identical, confirming that structured snapshot input imposes a negligible and consistent cost regardless of backend. The critical differentiators are completion token generation and its downstream impact on latency. GLM-5.1's consistently verbose outputs yield the richest explanations but incur latencies unsuitable for time-sensitive deployment. DS-V4-Flash achieves the lowest average latency but with substantial output-length volatility, which may complicate latency budgeting in production pipelines. MM-M2.5 offers the most predictable latency and token profile, making it the most deployment-friendly option when both reliability and resource predictability are required.

4.6.2. End-to-end latency and resource consumption

The per-call analysis focuses on the LLM module. To assess full-pipeline feasibility, we profile one 4500-sample NSL-KDD chunk and report stage-level latency and resource usage in Tables 5 and 6. The non-LLM part of SCALA-NIDS takes only ~2.2 s per chunk, including ~493.8 ms for open-world detection and ~1717.9 ms for multi-level updating. Peak GPU memory remains below 34 MB, and detection throughput exceeds 9,000 samples/s.

These measurements indicate that the detector, buffering, and adaptation modules are lightweight compared with LLM reasoning. There-

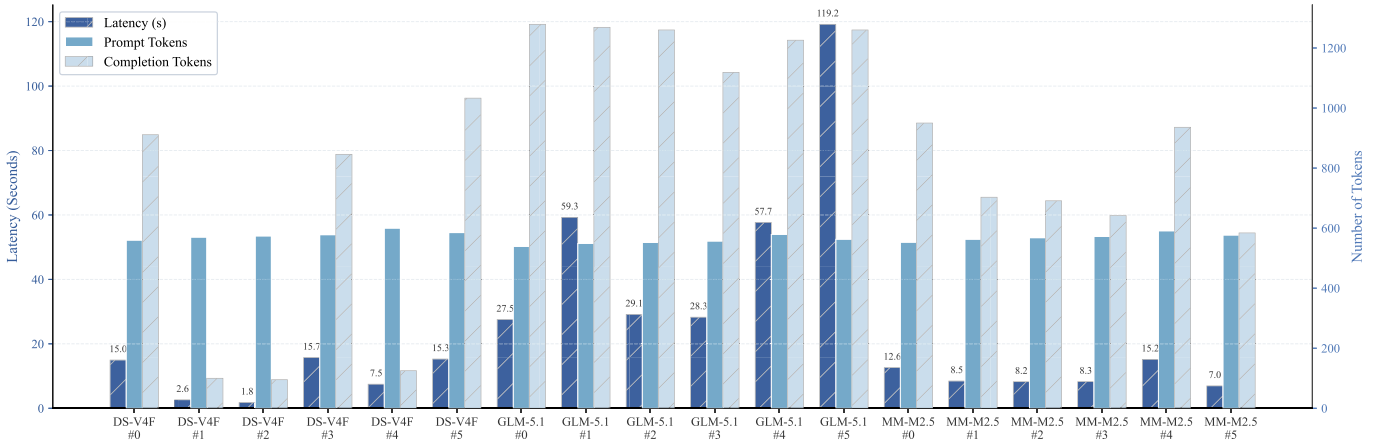


Fig. 5. Per-call resource consumption of the three models with accepted LLM-driven decisions on the NSL-KDD dataset. Each group reports latency (s, left axis), prompt tokens, and completion tokens (right axis) across six chunks (#0–#5). Kimi-K2.6 and Qwen3.5-397B are omitted because no LLM response from either backend is accepted as an executable strategy decision.

Table 5
End-to-end latency breakdown per chunk on NSL-KDD with 4500 samples.

Module	Average latency
① Open-world detector	≈ 493.8 ms
② LLM strategy reasoning	8–280 s
③ Multi-level updating	≈ 1717.9 ms
Total excluding LLM call	≈ 2211.7 ms
Total including LLM call	≈ 10–282 s

Table 6
Resource consumption on NSL-KDD.

Resource	Measured value
Peak GPU memory allocated	22.7 MB
Peak GPU memory reserved	33.6 MB
Peak CPU RSS	1746.2 MB
Detection throughput	9,121.4 samples/s

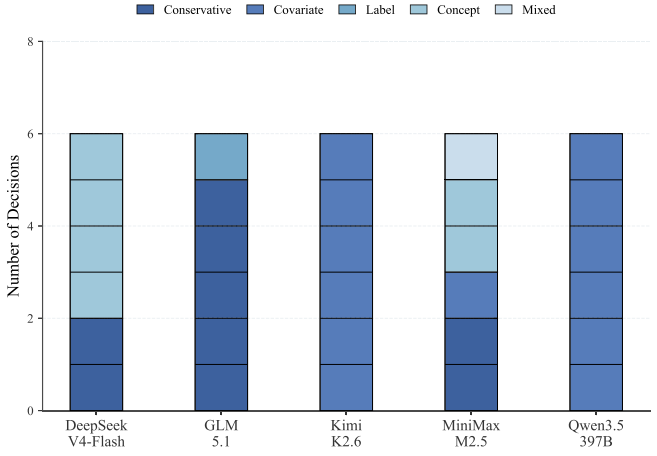


Fig. 6. Distribution of adaptation strategies selected by different LLMs. The strategies include conservative, covariate-shift-oriented, label-shift-oriented, concept-shift-oriented, and mixed strategies.

fore, the end-to-end wall-clock cost is governed mainly by the selected LLM backend: with a reliable low-latency model, the pipeline remains practical for chunk-level online adaptation, whereas timeout-prone backends shift the system into fallback mode and remove the benefit of LLM-specific reasoning.

4.6.3. Strategy distribution

Fig. 6 presents a stacked-bar comparison of strategy selections across the five LLM backends on the NSL-KDD dataset. Each model contributes six decisions, and the bars display the number of times each strategy type was selected. The five strategies—conservative, covariate, label, concept, and mixed—correspond to increasingly adaptive responses to distributional drift.

DeepSeek-V4-Flash exhibits a pronounced preference for the `concept` strategy, with four of six decisions classified as concept-related and one each falling into `conservative` and `covariate`. This concentration on concept-oriented adaptation suggests that DeepSeek interprets the KDD multi-signal indicators as primarily indicative of decision-boundary changes, warranting stronger consistency regularization and target-domain learning. Notably, it selects `mixed` in none of its six calls.

GLM-5.1 shows a markedly different emphasis: four of six decisions are classified as `covariate`, with only one `conservative` and one `concept`. This strong bias toward covariate-type adaptation indicates that GLM-5.1 attributes the observed distributional deviations primarily to feature-space shifts, triggering heuristic domain-alignment-focused adaptation. Its selection profile is the least diverse among the three fully or partially reliable models.

MiniMax-M2.5 produces the most diverse strategy distribution. Its six decisions span four categories: `conservative` (1), `covariate` (1), `label` (1), `concept` (2), and `mixed` (1). MiniMax-M2.5 is also the *only* model in the comparison whose LLM-generated outputs contain a `mixed` strategy call, indicating that it occasionally recognizes compound drift when other models assign a narrower label. This diversity aligns with its more verbose but balanced explanation profile discussed in Section 4.7.

Kimi-K2.6 and **Qwen3.5-397B** exhibit highly concentrated distributions dominated by `covariate` and `label` selections, with virtually no `concept` or `mixed` calls. The origin of this narrow profile, however, differs between the two models. Kimi-K2.6 experiences consistent timeouts on every call (see Table 4), producing zero prompt and zero completion tokens across all six chunks; consequently, its strategy sequence is determined entirely by the fallback module and represents the default behavior of rule-based decision-making under the KDD runtime snapshots. Qwen3.5-397B records limited token usage for one response attempt, but no response is accepted as an executable LLM-driven decision; therefore, its final strategy sequence is also determined by fallback for all six chunks and closely mirrors the fallback-only profile of Kimi-K2.6. The fallback module, guided by conservative heuristics, defaults

Table 7
LLM backend performance on NSL-KDD. Models marked with † are fallback-driven for all chunks.

Model	AUROC	Acc	Pre	Rec	F1
DeepSeek-V4-Flash	95.83	**90.42**	**89.79**	94.60	91.93
GLM-5.1	95.97	90.35	88.87	95.71	**91.97**
Kimi-K2.6†	95.87	89.71	86.19	**97.92**	91.64
MiniMax-M2.5	**95.98**	90.30	89.01	95.37	91.90
Qwen3.5-397B†	95.87	89.71	86.19	**97.92**	91.64

to covariate- or label-oriented strategies when domain discrepancy is detected but lacks the cross-signal reasoning to identify more complex drift patterns such as concept or mixed.

Summary. The strategy distribution reveals a clear correspondence between calling reliability and strategy diversity. The two fully reliable models (DeepSeek-V4-Flash and MiniMax-M2.5) produce the most nuanced distributions, albeit with opposite emphases—DeepSeek favors concept adaptation while MiniMax-M2.5 explores the full strategy spectrum including mixed. GLM-5.1, despite its successful calls, shows the least diverse reasoning, concentrated on a single drift interpretation. The fallback-only models (Kimi-K2.6 and Qwen3.5-397B) are structurally incapable of producing concept or mixed strategies, reflecting a fundamental limitation of rule-based decision pipelines.

4.6.4. Detection performance

Table 7 reports the detection performance on NSL-KDD. All LLM backends yield comparable results, with F1-scores ranging tightly from 91.64% to 91.97%. This narrow range is expected: all models operate under the same chunk configuration and fallback rules, and the fallback module produces deterministic, reproducible strategy decisions when the LLM is unavailable. Consequently, the performance is largely determined by the bounded adaptation pipeline rather than by individual LLM strategy differences.

Among the fully reliable models, DeepSeek-V4-Flash achieves the highest Accuracy (90.42%) and Precision (89.79%), while MiniMax-M2.5 achieves marginally higher AUROC (95.98% vs. 95.83%). GLM-5.1, despite its higher latency, attains the best F1-score (91.97%) and Recall (95.71%), suggesting that its more detailed outputs may deliver marginally more precise strategy decisions when calls succeed. Kimi-K2.6 and Qwen3.5-397B produce identical metrics (F1: 91.64%, Recall: 97.92%) because both rely entirely on the fallback module for all six chunks, which yields the same deterministic strategy sequence under the same KDD runtime snapshots. Compared with the fully reliable models, their notably higher Recall but lower Precision reflects the fallback module's tendency to favor more aggressive pseudo-label adoption when LLM reasoning is unavailable.

Overall, these results indicate that on the NSL-KDD dataset, calling reliability is the primary differentiating factor among LLM backends, while the final detection performance is largely comparable once the bounded adaptation pipeline is in place. The relatively narrow performance gap between SCALA-NIDS and the baselines on NSL-KDD can be attributed to two factors. First, NSL-KDD has been extensively studied and its feature space is relatively well-separated; even static baselines such as Random Forest and XGBoost achieve strong results, leaving limited room for adaptation-driven improvement. Second, the chunk-level drift on NSL-KDD is mild compared with UNSW-NB15, reducing the necessity for frequent and aggressive model updates. On UNSW-NB15, where the traffic distribution is more diverse and drift patterns are more pronounced, the advantage of adaptive strategies becomes more evident. Nevertheless, a reliable LLM advisor provides two qualitative advantages that fallback-only baselines cannot offer: (1) explicit, drift-indicator-grounded justifications for each strategy decision, and (2) the ability to adapt strategy selection to subtle context differences that rule-based fallback modules may overlook.

4.7. Strategy and explanation quality

This subsection evaluates whether the LLM advisor generates reasonable adaptation strategies and high-quality explanations grounded in observed drift indicators (RQ3).

4.7.1. Strategy reasonableness

For the evaluated NSL-KDD chunks, multi-signal indicators (e.g., `domain_acc` > 0.70 and elevated unknown ratios) suggest compound drift that spans both feature-level and decision-boundary shifts. The three models with successful LLM outputs respond to these signals with notably different interpretations. **DeepSeek-V4-Flash** predominantly selects the `concept` strategy, attributing the observed deviations to changes in the classification boundary and consistently triggering consistency regularization and target-domain learning. **GLM-5.1** interprets the same snapshots as primarily covariate drift, concentrating four of six decisions on feature-space adaptation. **MiniMax-M2.5** is the only model that outputs a `mixed` strategy call, recognizing that the KDD drift indicators collectively point to compound rather than single-source distributional change.

For models with frequent or total fallback dependence (Qwen3.5, Kimi-K2.6), strategy selection is largely dictated by rule-based modules that resolve to covariate- or label-oriented defaults. While effective in maintaining baseline performance, these fallback rules lack the explicit *reasoning chain* provided by an LLM, which is essential for auditability and trust in safety-critical NIDS.

4.7.2. Explanation quality

We evaluate explanation quality across six normalized metrics: text length (chars), word count, type-token ratio (lexical diversity), number of sentences (as a proxy for reasoning steps), drift-indicator mentions, and completion tokens. The results are summarized in Fig. 7. Only the three models with accepted LLM-driven decisions—DeepSeek-V4-Flash, GLM-5.1, and MiniMax-M2.5—are included in this comparison; Kimi-K2.6 and Qwen3.5-397B have no accepted LLM-driven decisions for any chunk and are therefore excluded.

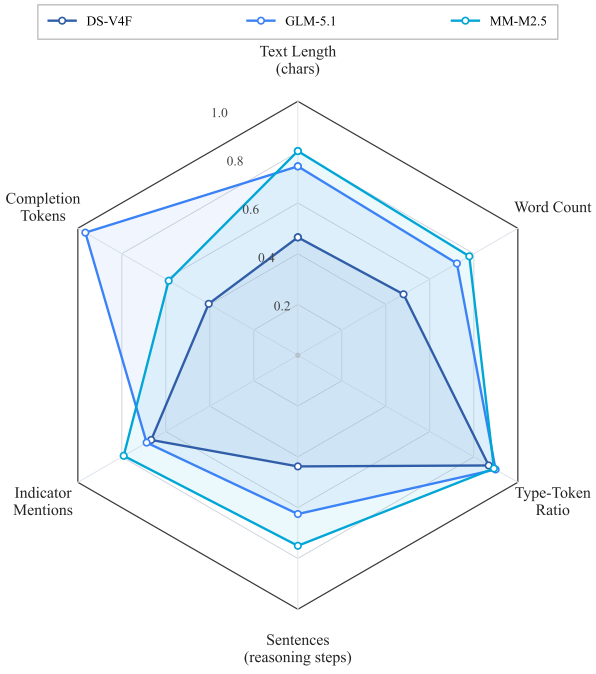


Fig. 7. Radar-chart comparison of LLM explanation quality. Metrics are averaged over successful calls and normalized by the maximum value. Only models with successful calls (DeepSeek-V4-Flash, GLM-5.1, MiniMax-M2.5) are shown.

Table 8
Ablation study on major framework components.

		UNSW-NB15				NSL-KDD			
Variant	Removed Component	Acc	Pre	Recall	F1	Acc	Pre	Recall	F1
Full Model	None	95.93	93.23	96.86	94.42	90.42	89.79	94.60	91.93
w/o Adaptation	Full adaptation module	93.88	85.32	86.80	86.05	88.68	88.85	89.04	88.94
w/o LLM	LLM guidance	94.55	90.80	93.36	92.04	89.40	86.50	96.74	91.33
w/o Safety Gate	Safety gate	94.13	90.58	93.80	92.16	89.23	85.93	97.19	91.21

Table 9
Ablation study on different loss terms.

		UNSW-NB15				NSL-KDD			
Variant	Removed Loss	Acc	Pre	Recall	F1	Acc	Pre	Recall	F1
Full Model	None	95.93	93.23	96.86	94.42	90.42	89.79	94.60	91.93
w/o Pseudo Loss	Pseudo-label loss	95.11	92.78	95.28	93.50	90.28	88.00	95.37	91.00
w/o Source Loss	Source-domain loss	95.01	92.39	95.09	93.22	89.99	87.33	95.70	90.78
w/o Consistency Loss	Consistency loss	94.75	92.20	94.83	92.98	89.54	86.81	95.56	90.43
w/o Domain Loss	Domain alignment loss	94.71	91.20	92.68	91.42	89.19	86.58	95.32	90.19

GLM-5.1 exhibits the largest overall coverage among the three models. It achieves the highest completion token count (normalized above 0.90), together with strong performance on text length, word count, type-token ratio, and indicator mentions, most of which fall near or above 0.70. Its sentence (reasoning step) count is comparatively lower, around 0.60, indicating that GLM-5.1 tends to produce longer, more densely packed explanations rather than decomposing its reasoning into numerous discrete steps.

MiniMax-M2.5 shows a more balanced profile. It attains high scores across text length, word count, type-token ratio, sentences, and indicator mentions, and notably surpasses GLM-5.1 in reasoning-step count (approximately 0.70). However, its completion token consumption is significantly lower than that of GLM-5.1 (around 0.55), confirming earlier observations from the calling-cost analysis: MiniMax-M2.5 achieves comparable coverage with substantially more compact outputs.

DeepSeek-V4-Flash presents the smallest overall footprint yet does not sacrifice crucial quality dimensions. Its type-token ratio approaches 0.80, the highest among the three models, reflecting superior lexical diversity despite shorter text length. Its indicator mention score (approximately 0.65) places it in the upper-middle range, and its substantially lower values on text length, word count, sentences, and completion tokens confirm that DeepSeek-V4-Flash produces concise, lexically rich explanations that remain grounded in drift-relevant indicators without excessive verbosity.

Summary of trade-offs. The three models exhibit distinct explanation profiles that reflect fundamentally different strategies. GLM-5.1 maximizes completeness at the cost of verbosity, making it suitable for applications where exhaustive documentation of each decision is required but latency and token cost are less constrained. MiniMax-M2.5 offers the most balanced coverage across all dimensions, achieving strong indicator grounding with moderate resource consumption. DeepSeek-V4-Flash favors conciseness and lexical precision, delivering the most economical yet diagnostically informative explanations. The choice among these profiles depends on the deployment context: latency-sensitive online NIDS pipelines benefit most from concise, indicator-focused reasoning, while forensic or post-hoc auditing scenarios may warrant the richer output produced by more verbose models.

4.7.3. Summary

The joint evaluation of RQ2 and RQ3 on the NSL-KDD dataset leads to three main conclusions.

(1) Reliability is paramount. Failed or rejected calls bypass LLM reasoning entirely and leave strategy decisions to deterministic fallback modules, so the practical value of any candidate LLM is determined first by whether it returns an accepted decision at all.

(2) No single reliable model dominates all dimensions. Among the three models with accepted outputs, GLM-5.1 leads in F1-score and explanation completeness; MiniMax-M2.5 offers the most balanced cost-quality profile; and DeepSeek-V4-Flash excels in conciseness and lexical diversity.

(3) Indicator-grounded justifications are a qualitative advantage of reliable LLM advisors. This advantage persists even when quantitative detection performance is similar across backends, because the LLM provides explicit, auditable reasoning that rule-based fallback modules cannot reproduce.

Since public NIDS benchmarks do not provide chunk-level drift-type ground truth, the LLM-predicted drift type is treated as an evidence-guided intermediate signal rather than a verified label. Its reliability is therefore assessed indirectly through semantic validation, indicator-grounded explanations, and downstream detection performance.

4.8. Ablation study

To analyze the contribution of each component, we design ablation experiments by removing one module at a time from the full framework. Tables 8 and 9 report the ablation results for major components and loss terms, respectively.

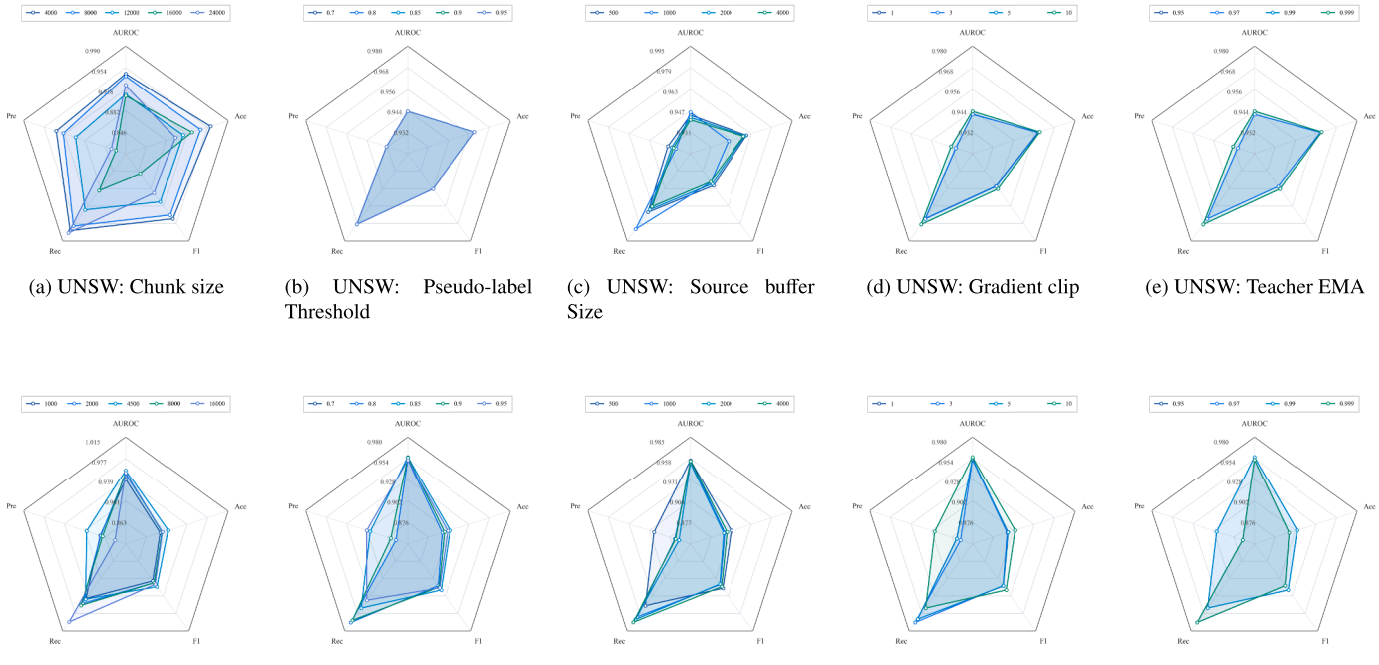
Component ablation. Removing the adaptation module causes the largest degradation (F1:-8.37 on UNSW,-2.99 on NSL-KDD), confirming it is the most critical component. Removing LLM guidance reduces F1 by 2.38 on UNSW and 0.60 on NSL-KDD, with precision dropping on both datasets, indicating that LLM reasoning helps filter false positives under domain shift. Removing the safety gate also reduces F1 and precision, as the model accepts more noisy updates without validation.

Loss ablation. Removing the domain alignment loss causes the largest degradation (F1:-3.00 on UNSW,-1.74 on NSL-KDD), confirming it is the most critical loss term. Removing the source-domain loss reduces F1 by 1.20 on UNSW and 1.15 on NSL-KDD, indicating that supervised source guidance remains important under domain shift. Removing the consistency loss also reduces F1 by 1.44 on UNSW and 1.50 on NSL-KDD, suggesting that consistency regularization helps stabilize predictions across domains. Removing the pseudo-label loss leads to a smaller but consistent drop in F1, showing that pseudo supervision further improves adaptation performance.

Overall, all proposed components and losses contribute complementary benefits. The full model achieves the most reliable performance by integrating adaptation, LLM guidance, safety gating, and balanced optimization objectives.

4.9. Hyperparameter sensitivity analysis

We evaluate the sensitivity of five key hyperparameters corresponding to the core components of the proposed framework: chunk



(a) UNSW: Chunk size (b) UNSW: Pseudo-label Threshold (c) UNSW: Source buffer Size (d) UNSW: Gradient clip (e) UNSW: Teacher EMA

(f) KDD: Chunk size (g) KDD: Pseudo-label Threshold (h) KDD: Source buffer Size (i) KDD: Gradient clip (j) KDD: Teacher EMA

Fig. 8. Hyperparameter sensitivity analysis using radar charts. Top row (a–e): UNSW-NB15 dataset. Bottom row (f–j): KDD dataset. Each radar chart shows the performance across five metrics (AUROC, Accuracy, F1, Recall, Precision) for different parameter values.

size, pseudo-label threshold, source buffer size, gradient clipping, and teacher EMA coefficient. The results are summarized in Fig. 8.

Data-level control. The pseudo-label threshold is robust across the tested range (<0.38% F1 variation); optimal values are 0.95 for UNSW and 0.85 for KDD. Smaller source buffers (500–1000) perform better on both datasets, suggesting that aggressive adaptation is preferred under severe distribution shift.

Model-level control. Gradient clipping shows minimal sensitivity (<0.24% F1 variation). The teacher EMA coefficient is evaluated in $\{0.95, 0.99, 0.995, 0.999\}$ (Fig. 8(e,j)); performance is stable across the entire range (<0.5% F1 variation), confirming that a default of 0.99 works well without fine-grained tuning.

Online protocol. Chunk size is the most sensitive parameter (up to 3.27% F1 drop on UNSW when suboptimal). Smaller chunks (4000–4500) consistently outperform larger ones, validating that frequent adaptation is crucial for handling rapid drift.

Overall, the framework is robust to most hyperparameters except chunk size, which should be tuned based on expected drift velocity.

5. Conclusion

We presented **SCALA-NIDS**, a framework that brings LLM-guided reasoning into online NIDS adaptation under a safety-constrained, closed-loop design. Rather than treating adaptation as an isolated model update, SCALA-NIDS orchestrates a "Detection–Response–Validation" cycle: it monitors runtime drift, consults an LLM for strategy-level advice, executes the chosen strategy through bounded, template-driven adaptation, and subjects every candidate update to post-hoc safety verification.

Across the UNSW-NB15 and NSL-KDD benchmarks, SCALA-NIDS outperformed representative online, continual, and traditional baselines on average accuracy and F1-score. The LLM-backend analysis on KDD revealed that calling reliability is the primary differentiator among candidate models, while the bounded adaptation pipeline keeps final detection performance robust even when the LLM backend varies. The explanation-quality study further showed that reliable LLMs can provide indicator-grounded, auditable justifications that rule-based fallback modules inherently lack.

That said, the current design involves two intentional trade-offs that point to natural directions for future extension. First, the LLM advisor is restricted to a fixed set of five high-level strategies, each mapped to a predefined parameter template. This bounded design is what makes the closed loop safe and reproducible; correspondingly, any drift behavior is mapped to the closest matching template within this strategy space. Future work may therefore explore a more expressive yet still bounded strategy space, in which new templates can be added under the same safety and validation contract. Second, the LLM is currently queried synchronously, once per chunk, before any adaptation is committed. This design keeps the advice tightly coupled to the most recent runtime evidence, and the per-chunk wall-clock cost is therefore dominated by the LLM call. Asynchronous or batched advisor invocation, in which the LLM is consulted in parallel with bounded adaptation rather than blocking it, is a promising direction to relax this coupling without weakening the safety gate.

Credit author statement

Xiaojie Qin led the conceptualization of the framework, designed the overall methodology, implemented the intrusion detection system, conducted formal analysis and experimental investigation, prepared the dataset curation, drafted the original manuscript, and created all visualizations. Qingjun Yuan contributed to the conceptualization and methodology design, validated the experimental results, supervised the research, administered the project, secured funding, and critically reviewed and edited the manuscript. Haopeng Fan contributed to the system implementation, validated the experimental outcomes, performed formal analysis, and assisted with dataset preparation and curation. Jing Tao contributed to the methodology design, validated the proposed framework, conducted formal analysis, and assisted with dataset preparation. Pinghui Wang contributed to the methodology design, provided computational resources and research infrastructure, and critically reviewed and edited the manuscript. Jihong Teng provided compu-

tational resources and researchinfrastructure, supervised the research, administered the project. Siqi Lu contributed to the system implementation, assisted with experimental investigation, and supported dataset preparation and curation. Yongjuan Wang provided computationalresources and research infrastructure and supervised the research.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

Bendale, A., & Boult, T. E. (2016). Towards open set deep networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)* (pp. 1563–1572). https://doi.org/10.1109/CVPR.2016.173

Breiman, L. (2001). Random forests. *Machine Learning*, *45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P. et al. (2020). Language models are few-shot learners. In *Advances in neural information processing systems* (pp. 1877–1901). (*vol. 33*).

Carlini, N., Tramer, F., Wallace, E., Jagielski, M., Herbert-Voss, A., Lee, K. et al. (2021). Extracting training data from large language models. In *30th USENIX security symposium* (pp. 2633–2650).

Chen, K., Yuan, Q., Meng, Q., Li, D., Zhu, Y., Yu, G., Wang, X., & Wang, Y. (2026). Robust intrusion detection in CPS: A pre-training-based multi-view feature collaboration and correlation analysis method. *Computer Networks*, *279*, 112129. https://doi.org/10.1016/j.comnet.2026.112129

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 785–794). https://doi.org/10.1145/2939672.2939785

De Lange, M., Aljundi, R., Masana, M., Parisot, S., Jia, X., Leonardis, A., Slabaugh, G., & Tuytelaars, T. (2022). A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *44*(7), 3366–3385. https://doi.org/10.1109/TPAMI.2021.3057446

European Union Agency for Cybersecurity (2024). ENISA threat landscape 2024. Technical Report European Union Agency for Cybersecurity (ENISA). https://www.enisa.europa.eu/publications/enisa-threat-landscape-2024

Ganin, Y., Ustinova, E., Ajakan, H., Germain, P., Larochelle, H., Laviolette, F. et al. (2016). Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, *17*(59), 1–35.

Gao, G., Niu, W., Gong, J., Gu, D., Li, S., Zhang, M., & Zhang, X. (2024). GraphTunnel: Robust DNS tunnel detection based on DNS recursive resolution graph. *IEEE Transactions on Information Forensics and Security*, *19*, 7705–7719.

Geng, C., Huang, S.-j., & Chen, S. (2021). Recent advances in open set recognition: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *43*(10), 3614–3631. https://doi.org/10.1109/TPAMI.2020.2981604

Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International conference on learning representations*.

Huang, S., Niu, W., Wang, R. et al. (2026). FlowCoPCL: Enhanced flow correlation attacks on Tor using patching and contrastive learning. *Tsinghua Science and Technology*, *31*(2), 745–759.

Karimian, M., & Beigy, H. (2023). Concept drift handling: A domain adaptation perspective. *Expert Systems with Applications*, *224*, 119946. https://doi.org/10.1016/j.eswa.2023.119946

Lee, K., Lee, K., Lee, H., & Shin, J. (2018). A simple unified framework for detecting out-of-distribution samples and adversarial attacks. In *Advances in neural information processing systems*. (*vol. 31*).

Liang, J., He, R., & Tan, T. (2025). A comprehensive survey on test-time adaptation under distribution shifts. *International Journal of Computer Vision*, *133*(1), 31–64. https://doi.org/10.1007/s11263-024-02181-w

Liang, J., Hu, D., & Feng, J. (2020). Do we really need to access the source data? Source hypothesis transfer for unsupervised domain adaptation. In *Proceedings of the 37th international conference on machine learning* (pp. 6028–6039). (*vol. 119*). PMLR.

Liang, S., Li, Y., & Srikant, R. (2018). Enhancing the reliability of out-of-distribution image detection in neural networks. In *International conference on learning representations*.

Liu, W., Wang, X., Owens, J. D., & Li, Y. (2020). Energy-based out-of-distribution detection. In *Advances in neural information processing systems* (pp. 21464–21475). (*vol. 33*).

Long, M., Cao, Y., Wang, J., & Jordan, M. I. (2015). Learning transferable features with deep adaptation networks. In *Proceedings of the 32nd international conference on machine learning* (pp. 97–105). (*vol. 37*). PMLR.

Long, M., Cao, Z., Wang, J., & Jordan, M. I. (2018). Conditional adversarial domain adaptation. In *Advances in neural information processing systems*. (*vol. 31*).

Manocchio, L. D., Layeghy, S., Lo, W. W., Kulatilleke, G. K., Sarhan, M., & Portmann, M. (2024). Flowtransformer: A transformer framework for flow-based network intrusion detection systems. *Expert Systems with Applications*, *241*, 122564. https://doi.org/10.1016/j.eswa.2023.122564

Meng, Q., Tao, J., Yuan, Q., Li, G., Wang, Y., Gao, B., & Lu, S. (2025). Detection of unknown attacks through encrypted traffic: A gaussian prototype-aided variational autoencoder framework. *IEEE Transactions on Information Forensics and Security*, *20*, 10652–10667.

Meng, Q., Yuan, Q., Shen, M., Lu, S., Li, G., Tao, J., Yu, Y., & Wang, Y. (2026). Fine-grained detection and analysis of unknown encrypted malicious traffic from mixed noisy labels. *IEEE Transactions on Dependable and Secure Computing*, (pp. 1–18). https://doi.org/10.1109/TDSC.2026.3697849

Moustafa, N., & Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems. In *2015 military communications and information systems conference (milCIS)* (pp. 1–6). https://doi.org/10.1109/MilCIS.2015.7348942

Niu, S., Wu, J., Zhang, Y., Chen, Y., Zheng, S., Zhao, P., & Tan, M. (2022). Efficient test-time model adaptation without forgetting. In *Proceedings of the 39th international conference on machine learning* (pp. 16888–16905). (*vol. 162*). PMLR.

Niu, S., Wu, J., Zhang, Y., Wen, Z., Chen, Y., Zhao, P., & Tan, M. (2023). Towards stable test-time adaptation in dynamic wild world. In *International conference on learning representations*.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C. et al. (2022). Training language models to follow instructions with human feedback. In *Advances in neural information processing systems* (pp. 27730–27744). (*vol. 35*).

OWASP Foundation (2025). OWASP Top 10 for large language model applications 2025. https://owasp.org/www-project-top-10-for-large-language-model-applications/

Quinlan, J. R. (1993). C4.5: Programs for machine learning. San Mateo, CA, USA: Morgan Kaufmann.

Scheirer, W. J., de Rezende Rocha, A., Sapkota, A., & Boult, T. E. (2013). Toward open set recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *35*(7), 1757–1772. https://doi.org/10.1109/TPAMI.2012.256

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E. et al. (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in neural information processing systems*. (*vol. 36*).

Shafee, S., Bessani, A., & Ferreira, P. M. (2025). Evaluation of LLM-based chatbots for OSINT-based cyber threat awareness. *Expert Systems with Applications*, *261*, 125509. https://doi.org/10.1016/j.eswa.2024.125509

Shen, M., Ji, K., Wu, J., Li, Q., Kong, X., Xu, K., & Zhu, L. (2024). Real-time website fingerprinting defense via traffic cluster anonymization. In *2024 IEEE symposium on security and privacy (SP)* (pp. 3238–3256).

Shen, M., Wu, J., Ai, J., Li, Q., Ren, C., Xu, K., & Zhu, L. (2025a). Swallow: A transfer-robust website fingerprinting attack via consistent feature learning. In *Proceedings of the 2025 ACM SIGSAC conference on computer and communications security* (pp. 1574–1588).

Shen, M., Wu, J., Ye, K., Xu, K., Xiong, G., & Zhu, L. (2025b). Robust detection of malicious encrypted traffic via contrastive learning. *IEEE Transactions on Information Forensics and Security*, .

Standards, N. (2023). Artificial intelligence risk management framework (AI RMF 1.0). Technical Report National Institute of Standards and Technology. https://doi.org/10.6028/NIST.AI.100-1

Sun, B., & Saenko, K. (2016). Deep CORAL: Correlation alignment for deep domain adaptation. In *European conference on computer vision workshops* (pp. 443–450). https://doi.org/10.1007/978-3-319-49409-8_35

Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. In *Proceedings of the 2009 IEEE symposium on computational intelligence for security and defense applications* (pp. 1–6). https://doi.org/10.1109/CISDA.2009.5356528

UCI Machine Learning Repository (1999). KDD Cup 1999 data. http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.HTML. Accessed: 2024-12-01.

Wang, D., Shelhamer, E., Liu, S., Olshausen, B. A., & Darrell, T. (2021). Tent: Fully test-time adaptation by entropy minimization. In *International conference on learning representations*.

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y. et al. (2024). Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, .

Wang, N., Chen, Y., Hu, Y., Lou, W., & Hou, Y. T. (2022a). FeCo: Boosting intrusion detection capability in IoT networks via contrastive learning. In *Ieee infocom 2022* (pp. 1409–1418). https://doi.org/10.1109/INFOCOM48880.2022.9796926

Wang, Q., Fink, O., Van Gool, L., & Dai, D. (2022b). Continual test-time domain adaptation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)* (pp. 7201–7211).

Wang, X., Yuan, Q., Yu, W., Meng, Q., Lu, S., He, W. et al. (2025). Entropy-regulated cross-modal generative fusion for multimodal network intrusion detection. *Information Fusion*, *126*, 103581.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F. et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in neural information processing systems* (pp. 24824–24837). (*vol. 35*).

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E. et al. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv:2308.08155.

Xu, H., Wang, S., Li, N., Wang, K., Zhao, Y., Chen, K. et al. (2025). Large language models for cyber security: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, . https://doi.org/10.1145/3769676

Yang, J., Zhou, K., Li, Y., & Liu, Z. (2024). Generalized out-of-distribution detection: A survey. *International Journal of Computer Vision*, *132*(12), 5635–5662. https://doi.org/10.1007/s11263-024-02117-4

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International conference on learning representations*.

Yuan, Q.-j., Meng, Q., Tao, J., Li, G., Fei, J., Lu, B., & Wang, Y. (2025). Multi-agent for network security monitoring and warning: A generative AI solution. *IEEE Network*, *39*, 114–121. https://api.semanticscholar.org/CorpusID:279347719

Yue, Y., Chen, X., Han, Z., Zeng, X., & Zhu, Y. (2022). Contrastive learning enhanced intrusion detection. *IEEE Transactions on Network and Service Management*, *19*(4), 4232–4247. https://doi.org/10.1109/TNSM.2022.3218843

Zhang, X., Zhao, R., Jiang, Z., Chen, H., Ding, Y., Ngai, E. C. H., & Yang, S.-H. (2025). Continual learning with strategic selection and forgetting for network intrusion detection. In *Ieee infocom 2025* (pp. 1–10). https://doi.org/10.1109/INFOCOM55648.2025.11044615

Zhang, X., Zhao, R., Jiang, Z., Sun, Z., Ding, Y., Ngai, E. C. H., & Yang, S.-H. (2024). AOC-IDS: Autonomous online framework with contrastive learning for intrusion detection. In *Ieee infocom 2024* (pp. 581–590). https://doi.org/10.1109/INFOCOM52122.2024.10621346